

UDIMY COURSE

Riccardo Loddo

September 27, 2025

Contents

1	Tutorial	4
1.1	Tutorial installazione	4
2	Introduzione	7
2.1	VirtualBox e macchine virtuali	7
2.1.1	Dataset MovieLens e introduzione a Hive	8
2.1.2	Hortonworks	10
2.2	Hadoop Ecosystem	10
2.2.1	YARN	11
2.2.2	MapReduce	11
2.2.3	PIG	12
2.2.4	Hive	12
2.2.5	Ambari	12
2.2.6	Mesos	12
2.2.7	Spark	12
2.2.8	Taz	13
2.2.9	HBase	13
2.2.10	Storm	13
2.2.11	Oozie	13
2.2.12	ZooKeeper	13
2.2.13	Data ingestion	13
2.3	External Data Storges	14
2.4	Query Engine	14
3	HDFS	15
3.0.1	Lettura di un file su HDFS	15
3.0.2	Scrittura su un file	15
3.0.3	NFS	16
3.0.4	Ambiente didattico vs Cluster	16
3.1	MapReduce	17
3.1.1	Gestione degli errori	19
3.1.2	Esempio concreto	20
3.2	Esempio MapReduce con 3 nodi	21
4	PIG	23
4.0.1	Comandi principali in PIG	24

5	Spark	25
5.1	Resilient Distributed Dataset (RDD)	26
5.1.1	Find the movie with the lowest average rating (with RDD's)	26
5.1.2	Trasformazioni sugli RDD	29
5.1.3	Azioni sugli RDD	29
5.1.4	Lazy Evaluation	29
5.2	Spark SQL	30
5.3	Spark using MLlib	31
5.3.1	Esercizio	33
6	Hive	34
6.1	Caricamento file su Hive	34
6.2	Schema On Read - Hive	35
6.2.1	LOAD DATA vs LOAD DATA LOCAL	35
6.2.2	Managed Tables vs External Tables	35
6.2.3	Partizionamento	36
6.2.4	Esercizio	36
6.3	Integrating MySQL and Hadoop (Sqoop)	37
6.4	Use Sqoop to import data from MySQL to HDFS/Hive	38
6.5	Use Sqoop to import data from Hive to MySQL	39
7	DataBase NON-Relazionali con Hadoop	41
7.1	API	41
7.1.1	HBase	42
7.1.2	Architettura di HBase - Partizionamento orizzontale	42
7.2	Import data in HBase	43
7.3	Cassandra	44
7.3.1	Il Teorema di CAP	44
7.3.2	Partizionamento Orizzontale - Cassandra (partition key)	45
7.3.3	Script Spark-Cassandra	45
7.4	MongoDB	47
7.4.1	Documenti e Collezioni in MongoDB	48
7.4.2	MongoDB Architettura	49
7.4.3	Secondary Delay	50
7.4.4	Sharding (partizionamento orizzontale)	50
7.5	Come scegliere un DBMS	50
7.5.1	Partizionamento verticale VS orizzontale	51
8	Controllo dei Cluster	53
8.1	YARN	53
8.1.1	Scheduler di YARN	53
8.2	Tez	54
8.2.1	Tez vs MapReducer	54
8.2.2	Hive con Tez	55
8.3	Mesos	55
8.3.1	Mesos vs YARN	55
8.4	ZooKeeper	55

8.4.1	Ruolo nei sistemi distribuiti	56
8.4.2	Znodes	56
8.4.3	Esempio pratico di ZooKeeper	56
8.5	Oozie	57
8.6	Zeppelin	58
8.7	Hue	58
9	Kafka & Flume	60

Chapter 1

Tutorial

1.1 Tutorial installazione

- Installare **VirtualBox**: software che permette di runnare una Virtual Machine sul pc.
- La seconda cosa da scaricare è l'immagine della macchina virtuale che utilizzeremo. Nel nostro caso si tratta della **Hortonworks Sandbox**, che contiene al suo interno un'installazione completa della **Hortonworks Data Platform (HDP)** e il tool di gestione grafico **Ambari**. Il file distribuito è in formato **.ova**, che racchiude sia il sistema operativo (Linux) sia tutte le impostazioni necessarie alla configurazione della VM. Avviando questa macchina virtuale otterremo un **cluster Hadoop a nodo singolo**, utile per esercitarsi e fare test: il nodo del cluster coincide infatti con la VM stessa. Per utilizzarla, è sufficiente importare il file **.ova** in VirtualBox. Basta aprire il file scaricato e VirtualBox proporrà automaticamente la procedura di importazione.
- A questo punto da VirtualBox è possibile avviare la VM semplicemente schiacciando su 'Avvia'. Al primo boot ci metterà un po' di tempo per scaricare e setappare tutte le impostazioni. Al termine del boot ci troveremo di fronte questa schermata (vedi figura 1.2).
- Ora che la VM è accesa è possibile accedere all'interfaccia Ambari. Per farlo basta accedere all'indirizzo `http://localhost:1080` da un browser.
- Verrete indirizzati alla pagina in figura 1.3, clickate su **LAUNCH DASHBOARD**. A questo punto inserite username: *maria_dev* e password: *maria_dev*. Questi username e password sono account definiti nel sistema Ambari e permettono di accedere al servizio web di Ambari e ai servizi Hadoop.

L'interfaccia Ambari si presenta come in figura 1.4

- FINE SETUP

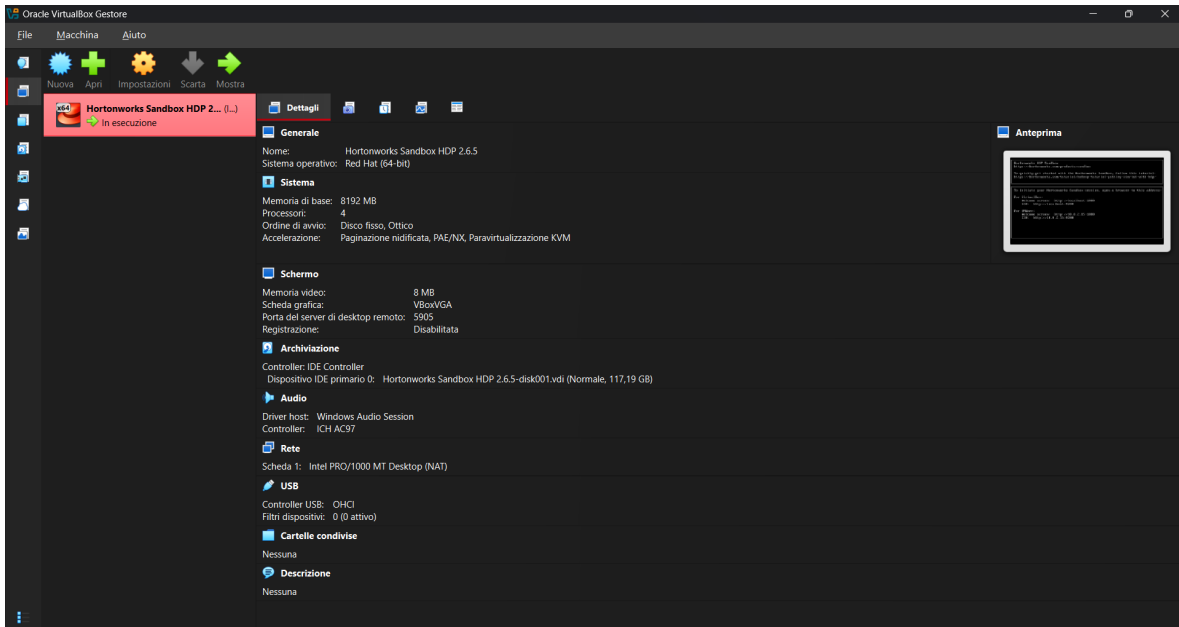


Figure 1.1: Esempio di immagine inserita in LaTeX

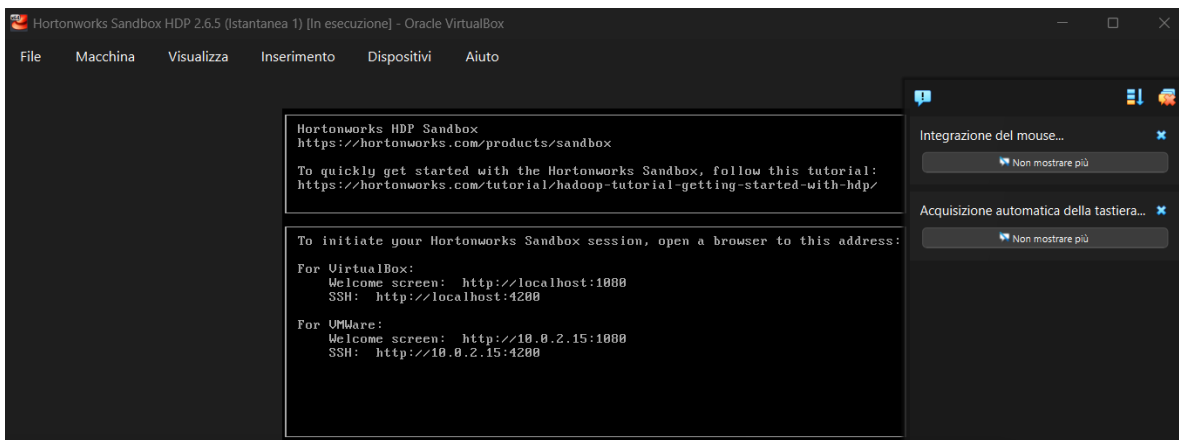


Figure 1.2: Schermata VM dopo il boot

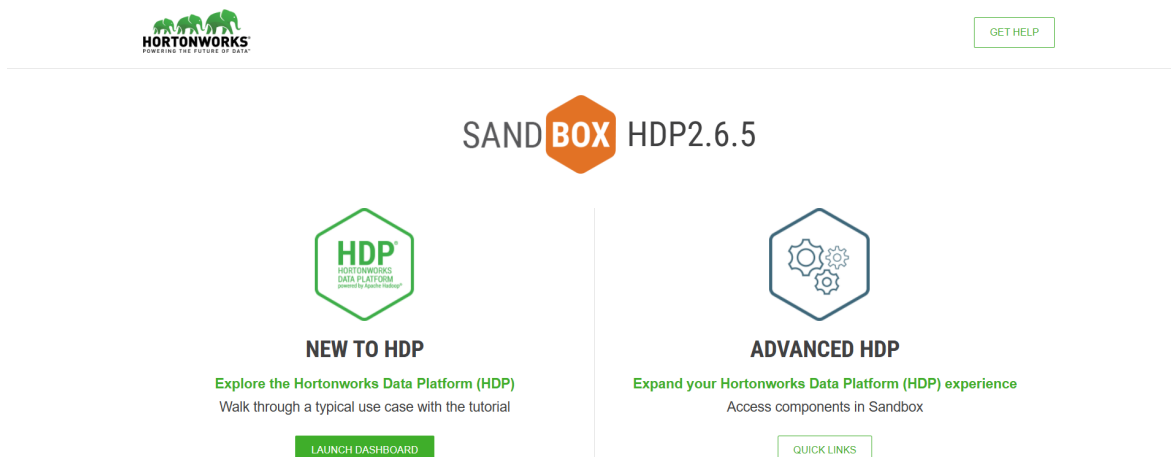


Figure 1.3: Pagina <http://localhost:1080>

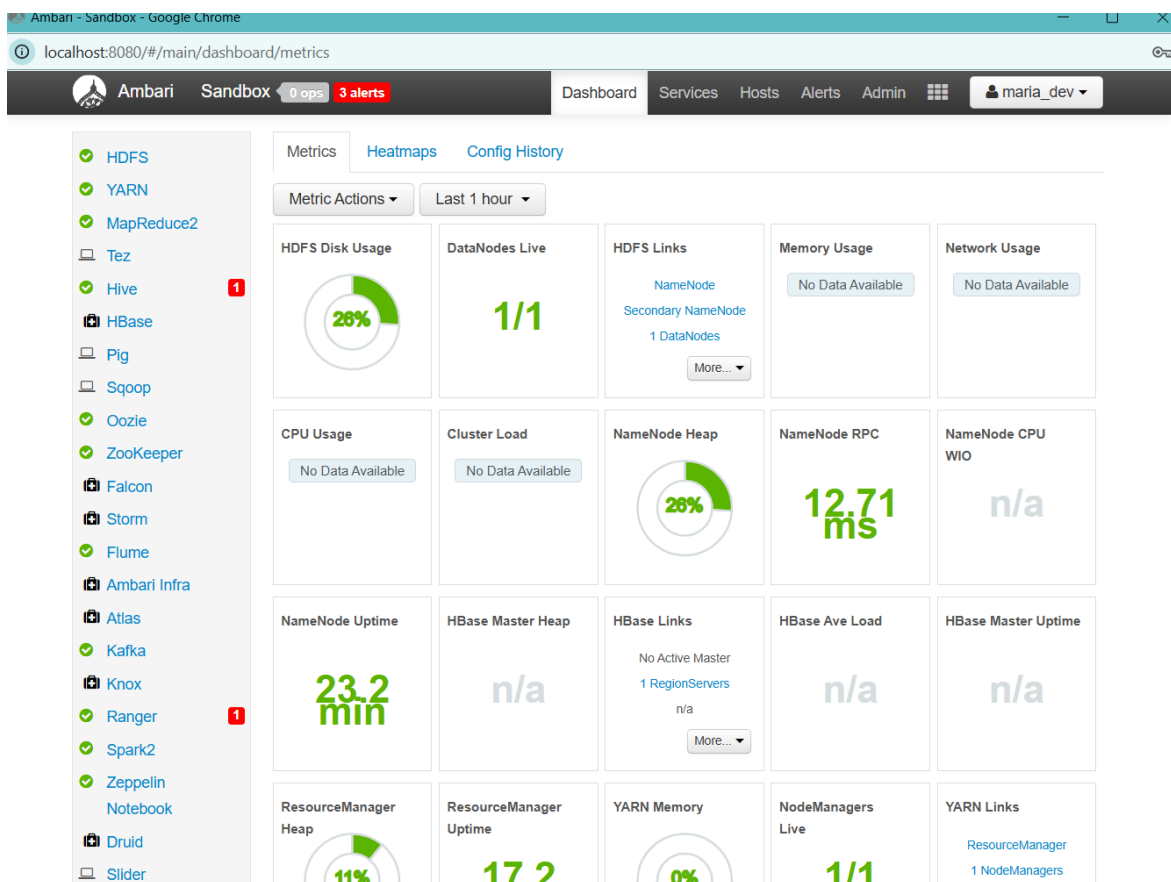


Figure 1.4: Ambari

Chapter 2

Introduzione

2.1 VirtualBox e macchine virtuali

VirtualBox è un software che consente di creare ed eseguire computer virtuali all'interno di un computer reale. Questi computer virtuali, denominati *Virtual Machine* (VM), sono a tutti gli effetti sistemi operativi che funzionano in modo isolato rispetto al sistema ospitante.

Il file `HDP_2.6.5_virtualbox_180626.ova` costituisce un pacchetto preconfigurato contenente una macchina virtuale già predisposta. Al suo interno è presente un sistema operativo Linux (verosimilmente CentOS), corredato dall'ecosistema Hadoop e da numerosi strumenti aggiuntivi quali Hive, HBase e Spark. L'importazione di tale file in VirtualBox consente di disporre di un ambiente di lavoro completo, senza la necessità di configurare manualmente i singoli componenti. All'avvio della macchina virtuale, eseguito tramite il comando *Esegui*, viene caricato il sistema operativo ospitato nella VM. Si apre quindi un terminale, analogo alla riga di comando tipica di Linux. Nella schermata iniziale vengono solitamente riportati alcuni indirizzi (`http://...`, `ssh user@...`, ecc.), che non rappresentano collegamenti casuali, bensì servizi già attivi all'interno della macchina virtuale. L'ecosistema Hadoop e i vari strumenti annessi sono infatti eseguiti come server nella VM, e tali indirizzi costituiscono i punti di accesso per interagire con essi dall'esterno, ossia dal computer *host*. Alcuni indirizzi corrispondono a interfacce web di amministrazione accessibili tramite browser, mentre altri permettono il collegamento via terminale tramite protocollo SSH.

L'accesso a determinati servizi, come ad esempio `http://localhost:1080`, richiede l'inserimento di credenziali (username e password). Tali credenziali non corrispondono ad account remoti, ma identificano esclusivamente utenti registrati all'interno del sistema della macchina virtuale. Generalmente:

- **Username:** utente definito nella configurazione della VM (ad esempio `maria_dev`, `root`, `hadoop`, ecc.);
- **Password:** chiave predefinita stabilita dal creatore della VM e fornita nel materiale del corso (in questo caso `maria_dev`).

Una volta effettuato l'accesso, viene presentato un **pannello di amministrazione** relativo all'ambiente Hadoop preinstallato nella VM. Al momento dell'avvio, i servizi principali (HDFS, YARN, Hive, Spark, HBase, ZooKeeper, ecc.) vengono inizializzati progressivamente. Durante questa fase compaiono numerosi avvisi (alert) in colore rosso o giallo, indicanti che i servizi

non sono ancora completamente attivi o non hanno stabilito le connessioni necessarie. Trascorsi alcuni minuti, la maggior parte degli avvisi scompare e lo stato dei servizi risulta segnalato in verde, a indicare che l'ambiente è operativo e pronto all'utilizzo.

2.1.1 Dataset MovieLens e introduzione a Hive

È stato utilizzato il dataset *MovieLens*, disponibile all'indirizzo <https://grouplens.org/datasets/movielens/>, scaricando la sezione *older datasets*. Il pacchetto compresso contiene diversi file, tra cui i dati di valutazione dei film (u.data). Ogni riga descrive l'azione di un utente che ha assegnato un punteggio ad un film in una determinata data. Ad esempio:

```
196 242 3 881250949
186 302 3 891717742
22 377 1 878887116
244 51 2 880606923
166 346 1 886397596
```

Le colonne rappresentano rispettivamente:

- identificativo dell'utente (UtenteID),
- identificativo del film (MovieID),
- valutazione espressa da 1 a 5 (rating),
- data del voto (in formato temporale Unix).

Un'altro file utile è u.item dove invece troviamo una mappatura degli ID dei film con i nomi dei film e una serie di altri metadati, in particolare la data di uscita, il link a IMDb e le informazioni sui generi cinematografici associati a ciascun titolo.

Il corso prevede l'importazione di questi dati nel cluster Hadoop, al fine di poterli interrogare mediante lo strumento **Hive**. Quest'ultimo non è un database relazionale¹ in senso stretto, bensì un framework che consente di eseguire query in linguaggio SQL su dati memorizzati in Hadoop, anche se questi non sono organizzati in tabelle relazionali tradizionali.

In altre parole, Hive non è un database relazionale, perché non gestisce i dati in modo “*centralizzato*” in tabelle vere e proprie, ma li lascia in file su Hadoop (HDFS). Inoltre, non c'è un motore relazionale che applica vincoli (primary key, foreign key, unique, ecc.). Hive traduce le query SQL in operazioni distribuite (originariamente MapReduce) eseguite sul cluster, permettendo quindi di gestire grandi quantità di dati in maniera scalabile pur mantenendo un'interfaccia simile a quella dei database classici.

Importando i dati in Hive e specificando il formato con cui sono separati, è possibile definire delle tabelle logiche che possono essere interrogate tramite linguaggio SQL. Ad esempio, per determinare quale sia il film più popolare, è sufficiente eseguire una query del tipo:

```
SELECT movie_id, COUNT(movie_id) AS ratingCount
FROM ratings
GROUP BY movie_id
ORDER BY ratingCount DESC;
```

¹Un database relazionale (RDBMS) è un sistema che memorizza i dati in tabelle composte da righe e colonne. Esempi di RDBMS sono OracleSQL, PostgreSQL, MySQL

La Fig. 2.1 mostra l'interfaccia di Hive all'interno della macchina virtuale.

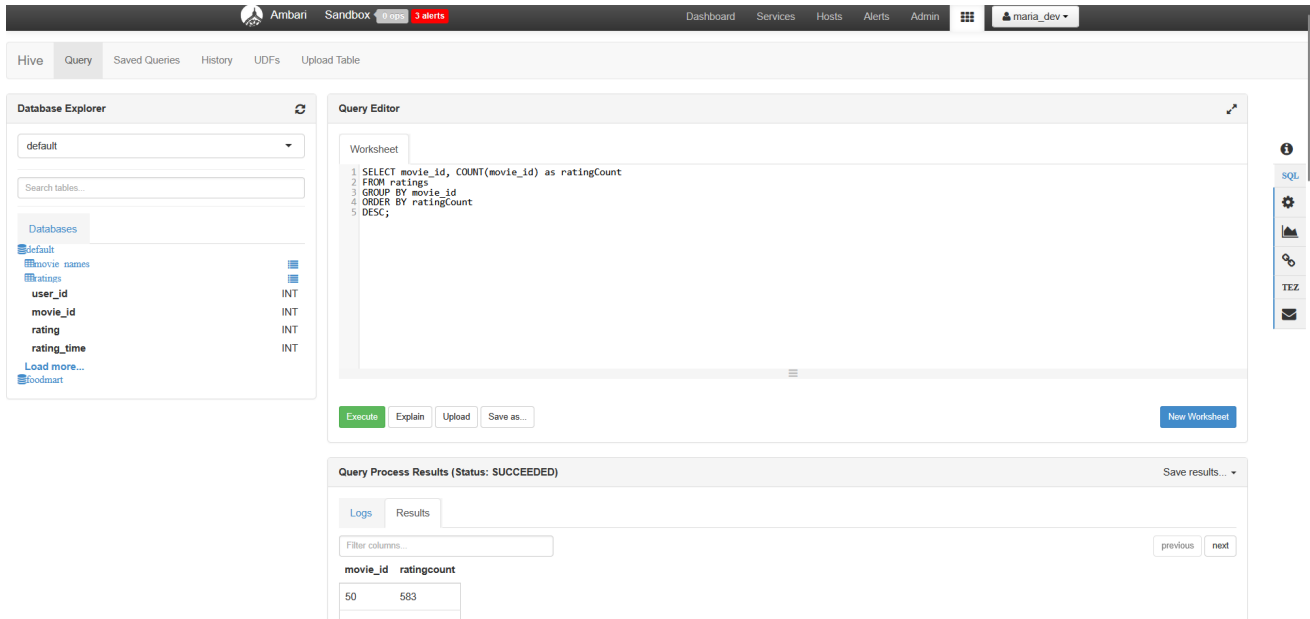


Figure 2.1: Interfaccia di Hive nella macchina virtuale.

Un aspetto rilevante è la possibilità di generare direttamente rappresentazioni grafiche dei risultati, come illustrato in Fig. 2.2.

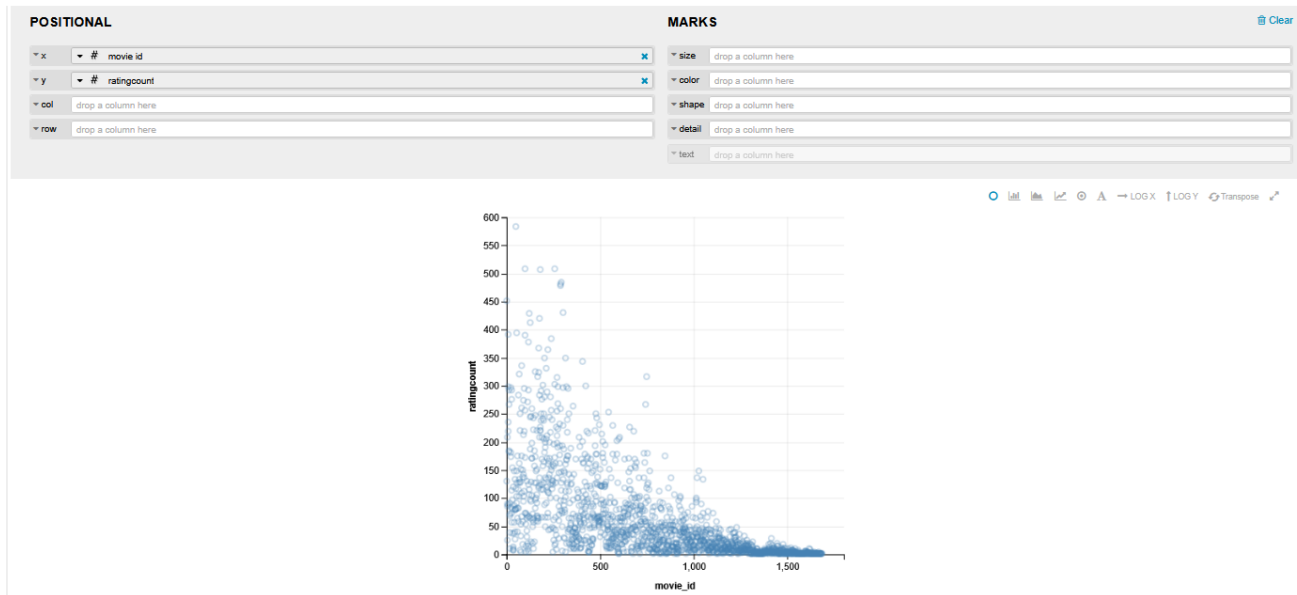


Figure 2.2: Grafico del numero di votazioni per ciascun `movie_id`.

Dall'analisi del grafico si osserva che i film con identificativo numerico più basso risultano maggiormente votati. Qualora l'assegnazione degli identificativi fosse avvenuta in ordine temporale, ciò indicherebbe che i film più datati presentano un numero maggiore di valutazioni.

2.1.2 Hortonworks

Il corso si basa sulla piattaforma dati **Hortonworks**, utilizzata come ambiente per l'esecuzione di operazioni pratiche sui Big Data direttamente sul computer locale. Hortonworks costituisce una distribuzione di Hadoop preconfigurata, che integra strumenti come Hive, HBase e Spark, consentendo di lavorare con grandi quantità di dati senza necessità di configurare manualmente ciascun componente.

È come per Linux: esistono Debian, Ubuntu, CentOS... tutte basate su Linux, ma con pacchetti già pronti, strumenti e supporto. Allo stesso modo, Hortonworks fornisce Hadoop già pronto, con dentro HDFS, YARN, Hive, Pig, Ambari e altri strumenti integrati, configurati per funzionare bene insieme.

È opportuno notare che Hortonworks si è fusa con l'azienda **Cloudera** all'inizio del 2019. A seguito della fusione è stata introdotta la **Cloudera Data Platform (CDP)**, una piattaforma disponibile sia in modalità cloud, eseguibile su server remoti, sia in modalità on-premise, utilizzabile nei data center locali. Nel contesto del corso, l'utilizzo di Hortonworks consente di simulare in maniera semplice e immediata un ambiente Hadoop completo su una macchina virtuale locale. Tenere a mente che:

- **Hadoop** → ecosistema open source
- **Hortonworks** → distribuzione di Hadoop pronta all'uso, include Hadoop + strumenti di gestione
- **Ambari** → tool di gestione incluso in Hortonworks

2.2 Hadoop Ecosystem

L'idea di base è che, quando si lavora con i Big Data, i file possono essere così grandi da non poter essere gestiti da un singolo computer. Per questo motivo è necessario suddividerli e distribuirli su più macchine, organizzate in un cluster. Sul nostro PC, ad esempio, vediamo un unico file system (cartelle e file), ma in realtà tutti i dati risiedono su un solo disco: quello del computer stesso. Ora immagina, invece, di avere 100 computer, ciascuno con il proprio disco: in questo caso i dati possono essere distribuiti tra queste macchine per essere gestiti in parallelo.

Senza un sistema distribuito, sarebbe un caos: non sapresti dove aprire i file, dovresti ricordarti che il pezzo 1 è sul PC A, il pezzo 2 sul PC B, ecc.

Con HDFS, invece, tu vedi sempre un unico "file system" (una grande cartella virtuale), ma dietro le quinte i file vengono spezzati in blocchi e sparsi sui vari dischi del cluster. HDFS che sta per Hadoop Distributed File System. Per sicurezza (fault tolerance), HDFS non salva un blocco una sola volta, ma lo replica (di solito 3 copie). Così, se un PC si rompe, i dati non si perdono. Se hai un CSV da 500 GB, un PC normale non riesce nemmeno ad aprirlo in Excel.

Con HDFS, invece, quel CSV viene spezzato e salvato su tanti computer → poi strumenti tipo Spark o Hive ci possono lavorare sopra in parallelo. **Osservazione:** nel continuo di questo corso noi useremo solo il nostro PC, quindi di fatto non stiamo suddividendo i file in cluster, ma concettualmente l'idea è questa. Se avessimo più macchine HDFS lo divide automaticamente in blocchi.

Un cluster Hadoop/HDFS non è un singolo computer, ma decine o centinaia di PC (**chiamati**

nodi). Ogni nodo ha il suo hard disk locale. HDFS usa questi dischi come pezzi di un disco gigante distribuito.

Quando salvi un file: HDFS lo spezza in blocchi (es. 128 MB ciascuno). Ogni blocco viene salvato su più nodi diversi, non sullo stesso PC. Per default, ogni blocco ha 3 copie (puoi configurare di più o di meno).

2.2.1 YARN

YARN è un pezzo fondamentale del puzzle Hadoop, ed è grazie a lui che Hadoop non è solo “**storage + MapReduce**” ma diventa un sistema molto più flessibile.

YARN sta per Yet Another Resource Negotiator. YARN gestisce le risorse dai cluster, ovvero quanta CPU, quanta RAM assegnare a ciascun PC per una operazione che vogliamo fare. Inoltre, decide quando e dove lanciare tale operazione. Infine ha un controllo sulla gestione di errori, ovvero che se un nodo muore YARN garantisce che il processo continui. In altre parole YARN è come se fosse il cervello che decide chi fa cosa; e lo fa sotto le quinte, ovvero noi non dobbiamo preoccuparci di nulla.

2.2.2 MapReduce

MapReduce è un modello di programmazione (e il framework relativo in Hadoop) pensato per elaborare grandi volumi di dati distribuiti su cluster di macchine. MapReduce è la parte storica della elaborazione dati di Hadoop. In particolare:

- **Map:** prendi i dati in input (spezzettati in blocchi da HDFS) e vengono elaborati in parallelo da più nodi del cluster. Ogni nodo applica una funzione **map** che trasforma l’input in coppie chiave - valore. Crea dunque una sorta di dizionario “*ridondante*”.
- **Reduce:** è il framework che si occupa di raccogliere tutte le coppie prodotte dalla fase map e le **raggruppa per chiave** e le passa ai **reducers**. Ogni reduce combina i valori associati alla stessa chiave per produrre un risultato finale. In altre parole, distrugge la *ridondanza* del dizionario creato da Map.

Ad esempio, supponiamo di avere un file gigantesco del tipo:

```
riga1: ciao come stai  
riga2: bene ciao  
riga3: ciao ciao amico
```

Ogni mapper legge una riga e produce coppie (**chiave, valore**).

$$\text{Mapper 1} \rightarrow (\text{ciao}, 1), (\text{come}, 1), (\text{stai}, 1)$$
$$\text{Mapper 2} \rightarrow (\text{bene}, 1), (\text{ciao}, 1)$$
$$\text{Mapper 3} \rightarrow (\text{ciao}, 1), (\text{ciao}, 1), (\text{amico}, 1)$$

Notiamo come la chiave “*ciao*” è presente più volte anche all’interno della singola riga. Ora si capisce bene perché chiamo il dizionario “*ridondante*”. Ora, dopo Map ma prima di Reduce c’è il

Shuffle & Sort che raggruppa tutte le coppie con la stessa chiave (parola) e le mette in ordine, ottenendo:

$$amico \rightarrow [1]bene \rightarrow [1] \quad ciao \rightarrow [1, 1, 1, 1] \quad come \rightarrow [1] \quad stai \rightarrow [1]$$

Ora entrano in gioco i reducer che sommano i valori associati a ciascuna chiave:

$$(ciao, 4), \quad (come, 1), \quad (stai, 1), \quad (bene, 1), \quad (amico, 1)$$

Da qui si estrae che il conteggio della parola *ciao* che è di 4.

2.2.3 PIG

Apache Pig è un framework sviluppato inizialmente da Yahoo! per rendere più facile scrivere programmi su Hadoop.

Al posto di scrivere complessi programmi MapReduce in Java (che è verboso e complicato), con Pig puoi usare un linguaggio più semplice chiamato Pig Latin (molto simile ad SQL).

Poi Pig traduce automaticamente lo script in job MapReduce che Hadoop esegue. Quidni PIG no è altro che un traduttore di linguaggio.

2.2.4 Hive

Apache Hive è un sistema che ti permette di fare query tipo SQL sui dati salvati in HDFS.

Nasce in Facebook: volevano che anche chi conosceva SQL (analisti, data scientist) potesse interrogare i Big Data senza imparare MapReduce o Pig.

Tu scrivi query HiveQL (simile a SQL), e Hive le traduce automaticamente in job MapReduce (o in Tez/Spark nei sistemi più moderni). .

2.2.5 Ambari

Apache Ambari è un pezzo “di servizio” nell’ecosistema Hadoop: non elabora dati come Hive o Pig, ma ti aiuta a gestire e monitorare il cluster. In altre parole è un interfaccia che permette di creare cartelle, caricare file ecc. senza dover scrivere tutto da terminale.

2.2.6 Mesos

Apache Mesos è un resource manager e cluster manager, molto simile (concettualmente) a YARN, ma più generico. È stato sviluppato all’Università di Berkeley e usato da grandi aziende come Twitter e Airbnb.

L’idea che c’è dietro Mesos è quella di non gestire solo Hadoop/MapReduce, ma qualunque tipo di applicazione distribuita (Hadoop, Spark, Kubernetes, Docker, servizi web...). Mentre YARN è pensato per Hadoo, Mesos, in generale, è pensato per qualsiasi tipo di carico di lavoro distribuito.

2.2.7 Spark

Apache Spark, che è uno dei componenti più importanti e moderni dell’ecosistema Hadoop. Spark è un motore di elaborazione dati veloce per Big Data.

È nato per superare i limiti di MapReduce, soprattutto in termini di velocità e facilità d’uso.

2.2.8 Taz

Analogo a Spark...migliore di mapreduce.

2.2.9 HBase

Apache HBase è un database distribuito NoSQL basato su Hadoop. È progettato per gestire enormi quantità di dati non strutturati o semi-strutturati.

2.2.10 Storm

Apache Storm è un motore di elaborazione dati in tempo reale (streaming). Nasce per processare flussi di dati continui, diversamente da MapReduce o Hive, che lavorano su batch (file già salvati). È molto usato per analisi real-time, monitoraggio, alerting, elaborazioni su log, social media, IoT.

2.2.11 Oozie

Apache Oozie è un workflow scheduler per Hadoop. Serve a automatizzare e coordinare i job che girano nel cluster. Ad esempio, se dobbiamo organizzare un hadoop, molto complicato con diverse operazioni, oozie permette di pianificare, controllare ogni singola azione.

2.2.12 ZooKeeper

Apache ZooKeeper è un servizio centralizzato per la gestione di configurazioni, sincronizzazione e naming nei sistemi distribuiti.

Serve quando hai più nodi che devono collaborare, e vuoi evitare conflitti, perdita di dati o comportamenti incoerenti.

2.2.13 Data ingestion

Entrambi Sqoop e Kafka rientrano nella data ingestion, cioè nella fase in cui i dati entrano nel tuo ecosistema Big Data, ma lo fanno in modi diversi. Data ingestion = il processo di raccolta, importazione e caricamento dei dati da fonti esterne verso il tuo cluster Hadoop o Big Data platform. Abbiamo due tipi di ingestion: Batch ingestion: grandi quantità di dati caricati a intervalli regolari Ad esempio: ogni notte importi i dati del giorno passato da un database Oracle a HDFS. In questo caso lo strumento tipico è Sqoop.

Mentre per dati real time che arrivano continuamente si utilizza Kafka che è una piattaforma di streaming distribuita. In Fig(2.3) è possibile osservare la struttura e la gerarchia dell'ecosistema di Hadoop.

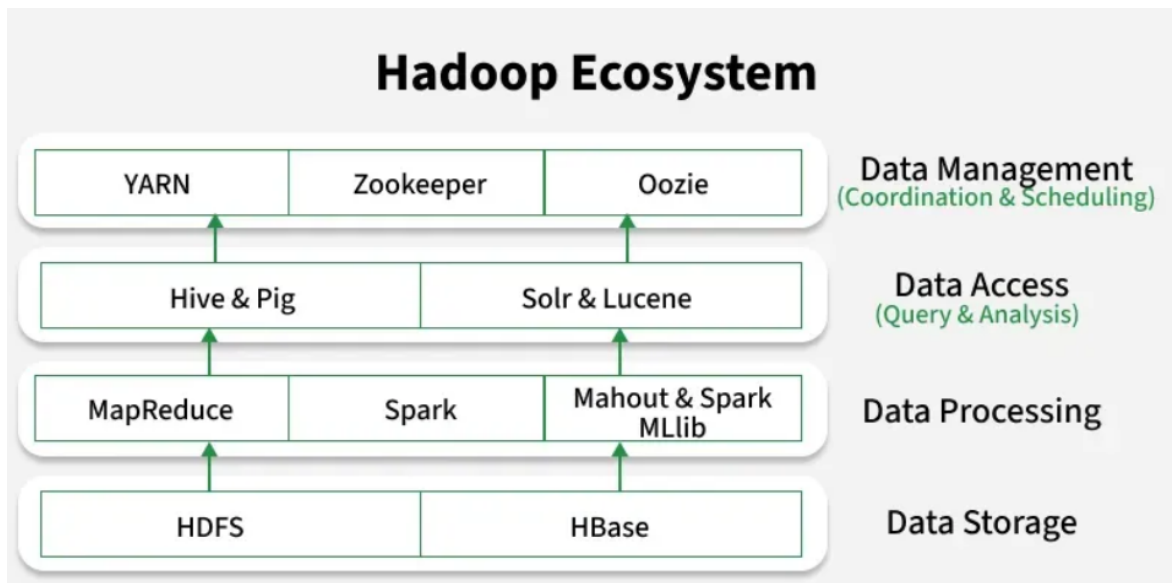


Figure 2.3: Hadoop Ecosystem

2.3 External Data Storages

HDFS = il file system distribuito di Hadoop, dove i dati sono salvati in blocchi su diversi nodi del cluster, con replicazione per tolleranza ai guasti. Tuttavia, oltre a fare diversi salvataggi, Hadoop può interagire con database esterni, cioè sistemi di storage non nativi Hadoop. Ad esempio per tipi di storage Relazionali MySQL, Oracle. Per NoSQL Cassandra, MongoDB.

2.4 Query Engine

Un query engine è un motore che esegue query sui dati, traducendo le richieste dell'utente (tipicamente SQL o linguaggi simili) in operazioni sui dati distribuiti. L'obiettivo è rendere l'accesso ai dati rapido, flessibile e trasparente, indipendentemente da dove siano salvati (HDFS, HBase, S3, database esterni). Gli esempi principali sono: Apache Drill, Impala (UE = Cloudera's Unified Engine), Apache Phoenix, Apache Zeppelin.

Chapter 3

HDFS

HDFS (Hadoop Distributed File System) è il **file system** distribuito di Hadoop, progettato per la gestione di grandi file di dati. La sua logica si basa su:

- **Spezzettamento dei file:** i file vengono suddivisi in blocchi e distribuiti all'interno del cluster.
- **Distribuzione dei dati:** i blocchi sono replicati su più macchine, garantendo sia l'elaborazione parallela sia la tolleranza ai guasti.
- **Replica dei blocchi:** ogni blocco viene copiato su nodi diversi, così se una macchina si guasta il dato è comunque disponibile altrove.

3.0.1 Lettura di un file su HDFS

HDFS è strutturato in due ruoli principali:

- **NameNode:** è il “cervello” del file system. Tiene traccia della posizione di ogni blocco di ogni file nel cluster.
- **DataNode:** sono i nodi che contengono fisicamente i blocchi di dati.

Ogni blocco di un file viene replicato su più DataNode (tipicamente 3).

Supponiamo di voler leggere `file.txt` in HDFS. Il processo è:

1. Richiesta al NameNode: Il client (programma o utente) chiede: voglio leggere il file `file.txt`. NameNode risponde ok, il file è composto da questi blocchi (Blocco 1 → DataNode A, B, C...)
2. Accesso ai DataNode: ora il client sa dove sono i blocchi. Si collega direttamente ai DataNode che contengono i blocchi richiesti e legge i dati in parallelo, nodo per nodo.

3.0.2 Scrittura su un file

Per la scrittura il processo è un po più complicato. Siccome i file sono salvati in diversi blocchi, allora se modifico una parte di blocco (salvato in tre parti diverse) automaticamente tutte le altre copie si dovranno "aggiornare". Quindi i vari nodi devono parlare tra di loro. Formalizziamo il processo:

1. Il client chiede al NameNode: “*Voglio scrivere questo file in HDFS*”. Il NameNode decide su quali DataNode andranno i blocchi e le loro repliche.
2. **Il NameNode risponde al client:** Ok, il primo blocco deve andare su DataNode A, B e C.
3. **Il client scrive al primo DataNode (ad esempio A):** il client non scrive direttamente su tutti i DataNode, ma su uno solo della catena.
4. **Pipeline di repliche:** DataNode A riceve i dati dal client che li salva localmente. Successivamente li inoltra al DataNode B e poi al C.
5. **Conferma al client:** Quando tutte le repliche sono state salvate correttamente, i DataNode confermano al client tramite il NameNode.

3.0.3 NFS

NFS sta per: Network File System. È un protocollo di rete sviluppato da Sun Microsystems (anni '80) che permette a un computer di accedere a file remoti come se fossero locali. In pratica un server esporta una cartella, e i client possono montarla e leggerla/scriverla come se fosse sul proprio disco.

3.0.4 Ambiente didattico vs Cluster

Il nostro è un ambiente didattico ovvero, non siamo su un vero cluster (cioè non ho tante macchine) ma ne ho solo una, il mio PC. Quando entro su HDFS con la virtual box, tutto è salvato su un unico PC/VM. Quindi si ha un NameNode e un DataNode (a volte il NameNode e il DataNode stanno persino nello stesso processo). Non c'è possibilità di “replicare” su più macchine, il file resta fisicamente su un unico disco. L'interfaccia però si comporta come se fosse un cluster: ti fa vedere HDFS con cartelle, utenti ecc.

Se volessimo davvero essere in un cluster, allora servirebbero più PC, tutti collegati con la stessa rete tramite IP/hostname. Su un PC installi il NameNode, mentre sugli altri i DataNode.

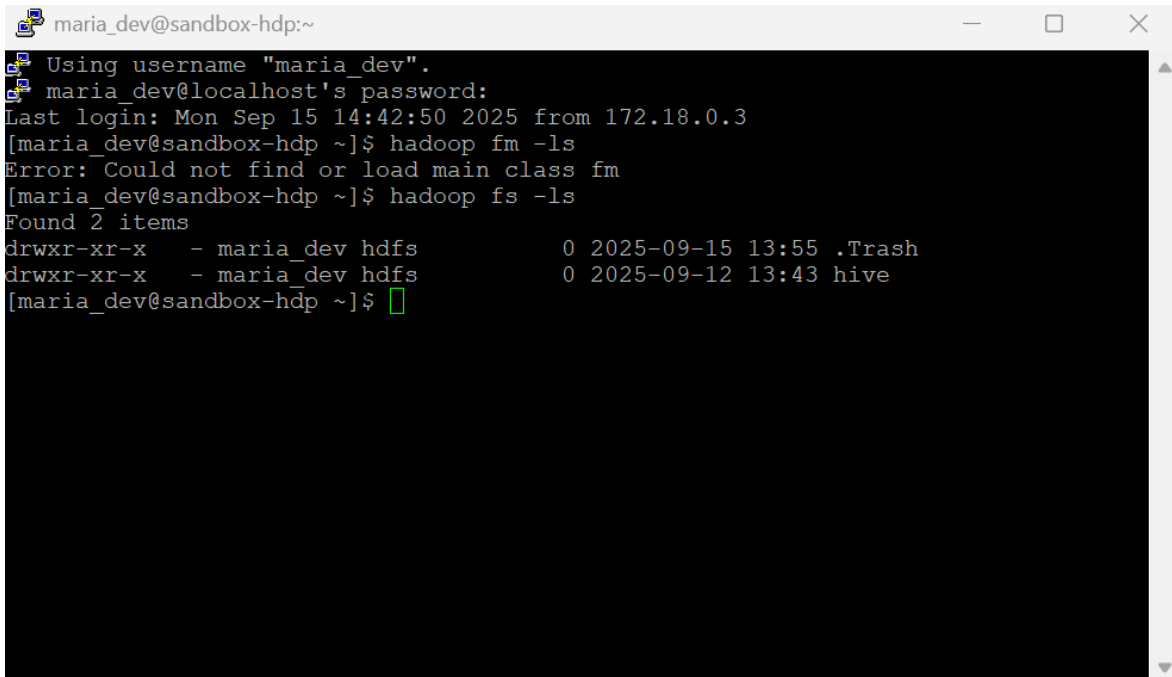
Su HDFS ho caricato, nell'utente `maria_dev` (vedi Fig. (3.1)) i due file dei film e volendo si possono selezionare entrambi e concatenare (ovvero vedere assieme). Ora rifaccio lo stesso

Name >	Size >	Last Modified >	Owner >	Group >	Permission
.Trash	--	2025-09-15 15:55	maria_dev	hdfs	drwxr-xr-x
hive	--	2025-09-12 15:43	maria_dev	hdfs	drwxr-xr-x

Figure 3.1: Interfaccia HDFS

ma da linea di comando. Ricordo infatti che HDFS ha una sua interfaccia grafica, ma si possono fare ugualmente da terminale. Per farlo dobbiamo anzitutto connetterci con la nostra

istanza. Lo facciamo con PUTTY, dove alla fine otteniamo una schermata come in Fig (3.2). Per creare una directory usiamo il comando: `hadoop fs -mkdir ml-100k`. Con il comando



```

maria_dev@sandbox-hdp:~
Using username "maria_dev".
maria_dev@localhost's password:
Last login: Mon Sep 15 14:42:50 2025 from 172.18.0.3
[maria_dev@sandbox-hdp ~]$ hadoop fm -ls
Error: Could not find or load main class fm
[maria_dev@sandbox-hdp ~]$ hadoop fs -ls
Found 2 items
drwxr-xr-x  - maria_dev hdfs      0 2025-09-15 13:55 .Trash
drwxr-xr-x  - maria_dev hdfs      0 2025-09-12 13:43 hive
[maria_dev@sandbox-hdp ~]$
```

Figure 3.2: Interfaccia Putty: il comando `hadoop fs -ls` è un’istruzione della Hadoop File System (HDFS). Hadoop richiama il client di Hadoop, `fs` sta per file system, cioè il sottocomando che permette di interagire con l’HDFS. `-ls` mostra l’elenco dei file e delle directory (analogo a `ls` su Linux, ma applicato all’HDFS).

`wget http://media.sundog-soft.com/hadoop/ml-100k/u.data` è stato scaricato nella directory corrente il file `u.data` da un server remoto. Successivamente, con il comando `ls -la`, è stato elencato il contenuto della cartella, in modo da verificare la presenza del file appena scaricato insieme ai dettagli relativi (permessi, dimensioni, proprietario e data di modifica). Per caricarlo su HDFS, usiamo il comando: `fs -copyFromLocal u.data ml-100k/u.data`. Per rimuoverlo basta fare: `hadoop fs -rm ml-100k/u.data`. Per rimuovere la cartella stessa: `hadoop fs -rmdir ml-100k`.

3.1 MapReduce

MapReduce è il modello di programmazione utilizzato da Hadoop per eseguire elaborazioni distribuite all’interno del cluster. Il suo funzionamento si basa sulla suddivisione dei dati in partizioni, che vengono poi processate in due fasi distinte: *Map* e *Reduce*. In breve il mapping trasforma/elabora i dati in parallelo mentre la fase di reducing consiste nell’aggregare i risultati del mapping in uno unico.

Fase Map

La fase di mapping consiste nell’applicare una funzione scritta dall’utente (in Java, Python o altri linguaggi) che assegna ad ogni *chiave* un *valore*.

Vogliamo capire quanti film ha valutato l'utente ID 1, quanti l'ID 2... Vogliamo farlo con mapreduce. la prima cosa che dobbiamo fare è trasformare i dati di input in dati grezzi che abbiamo nel nostro set di dati in un formato che abbia senso per noi. Quindi il nostro mappatore deve prendere ogni riga di dati di input che sarà una valutazione di un singolo film e la trasformerà in una coppia di valori chiave. Quindi la nostra chiave sarebbe l'id utente in questo esempio giusto.

E il valore sarebbero i film che stiamo aggregando. Ovvero avremmo come una sorta di dizionario del tipo:

$$\{196 : 242, 186 : 302, 196 : 307, 166 : 242, 186 : 423, 186 : 234, \dots\}$$

dove 196 è l'ID utente ed 242 è l'ID del film. Ora, senza scrivere una sola riga di codice, MapReduce farà questo passo successivo automaticamente: chiamato **shuffle** e **sort**, ed è qui che aggrega tutti i valori dati per ogni chiave univoca per te. E inoltre ordina quelle chiavi che potrebbero essere utili in alcune situazioni di ricerca. Ovvero il nostro dizionario diventa:

$$\{166 : 242, \dots, 186 : (302, 423, 234), \dots, 196 : (242, 307)\}$$

Ovvero l'utente 166 ha valutato solo il film 242. L'utente 186 ha valutato 3 film, e così via. Quindi tutte le valutazioni per l'id utente 186 potrebbero effettivamente provenire da mapper su computer completamente diversi.

Il potere di MapReduce è che aggatherà tutti questi valori per l'intero cluster e li metterà tutti insieme in un punto in cui è possibile elaborarli insieme. E questo è esattamente ciò che fa il tuo riduttore.

Fase Reduce

Il *Reducer*, anch'esso definito dall'utente, prende in input le chiavi e i loro insiemi di valori, per poi processarli e restituire un risultato aggregato. in questo esempio, forse riceviamo solo un elenco di film chiamati film e chiamiamo qualche operatore `len()` su di esso per ottenere la lunghezza di tale elenco, ottenendo:

$$\{166 : 1, \dots, 186 : 3, \dots, 196 : 2, \dots\}$$

In Fig(3.3) possiamo vedere un esempio completo di MapReduce.

Ricapitolando: ogni blocco (split) viene elaborato in parallelo da un mapper. Il mapper quindi non lavora sul file intero, ma solo sul suo pezzo di dati. Alla fine produce coppie chiave-valore. Tutte le coppie chiave-valore prodotte dai mapper vengono raggruppate per chiave. Qui Hadoop si occupa di raccogliere i dati dai vari nodi e inviarli ai reducer giusti. È il momento in cui i "pezzi sparsi" si ricombinano in base alla chiave, non come concatenazione di file. Ogni reducer riceve tutte le coppie relative a una certa chiave (anche se provenivano da blocchi diversi). Qui si esegue l'aggregazione finale (conteggi, somme, medie, ecc.). MapReduce è scritto in Java (`map()` ed `reduce()`) e non è un linguaggio così semplice come Python. **Hadoop Streaming** è un meccanismo che ti permette di scrivere mapper e reducer in qualsiasi linguaggio (Python, Perl, Bash, ecc.). L'idea è molto semplice: Hadoop tratta il tuo programma come un processo esterno e gli passa i dati tramite lo standard input (stdin). Il tuo programma elabora i dati e restituisce il risultato tramite lo standard output (stdout). Hadoop poi raccoglie questo output e lo reinserisce nel flusso di MapReduce.

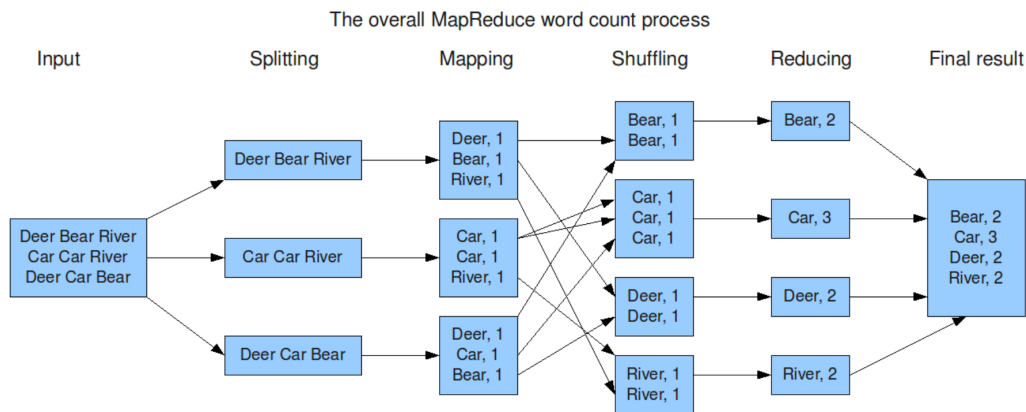


Figure 3.3: MapReduce World. Input non è l'intero file originale (che magari è da 10 GB), ma già un blocco di HDFS (per esempio 128 MB) che si trova su un nodo del cluster.

3.1.1 Gestione degli errori

In un'applicazione Hadoop MapReduce, la gestione degli errori è progettata per garantire che l'elaborazione dei dati continui anche in caso di guasti parziali. Consideriamo un esempio pratico: Supponiamo di avere un cluster Hadoop con diversi nodi e un job MapReduce che deve elaborare milioni di record. Il job è suddiviso in più task: mapper e reducer. Ogni mapper elabora solo una parte dei dati.

Durante l'esecuzione:

- Se un singolo task (mapper o reducer) fallisce o rimane sospeso, il master dell'applicazione lo rileva e tenta di riavviarlo. Il master dell'applicazione in Hadoop MapReduce è il componente centrale che coordina l'esecuzione di un job MapReduce. Per semplificare, possiamo paragonarlo a un direttore d'orchestra: non esegue direttamente il lavoro di elaborazione dei dati, ma controlla chi fa cosa, quando e come. Spesso è chiamato ApplicationMaster (AM).
- Il task fallito viene preferibilmente riavviato su un nodo diverso, evitando il nodo che ha causato l'errore.
- Questo significa che non è necessario ricominciare tutto il job dall'inizio, ma solo il task specifico.

Un caso più complicato è quando il master dell'applicazione stesso va giù. Il master è gestito dal ResourceManager di Hadoop (ovvero YARN). Se il master fallisce, YARN può riavviarlo automaticamente su un altro nodo.

Infine, se un intero nodo del cluster fallisce: tutti i task eseguiti su quel nodo vengono considerati falliti. Hadoop li riparte su altri nodi disponibili, dato che ogni task gestisce solo un sottoinsieme dei dati. In questo modo, anche la perdita di un nodo non interrompe

Si può tollerare anche il fallimento del ResourceManager: Hadoop può essere configurato con alta disponibilità tramite ZooKeeper. ZooKeeper tiene traccia del ResourceManager attivo e, se questo va giù, reindirizza i task verso un ResourceManager di backup, così l'intero sistema diventa molto più resistente ai guasti.

```

# Definisco i nomi delle colonne
nomi_colonne = ['user_id', 'item_id', 'rating', 'timestamp']

# Leggo il file separato da tab
data = pd.read_csv("u.data", sep='\t', header=None, names=nomi_colonne)

# Controllo i primi dati
print(data.head())

def mapper_get_ratings(self, _, line):
    (userID, movieID, rating, timestamp) = line.split('\t')
    yield rating, 1

def reducer_count_ratings(self, key, values):
    yield key, sum(values)

```

Figure 3.4: Funzione di mapping e reducer.

3.1.2 Esempio concreto

Ritorniamo al nostro set di dati dei film. quale sia la distribuzione di questi voti e quanto siano esigenti le persone, almeno quanto lo erano nel 1998, quando è stato creato questo set di dati. Quindi la prima cosa che il nostro mappatore deve fare è estrarre le informazioni che ci interessano e buttare via quelle che non ci interessano.

In questo caso, l'unica cosa che ci interessa sono le valutazioni stesse, perché vogliamo solo contare quanti tre o quanti uno ci sono. Lo facciamo con tutte le ridondanze, non vado a raggruppare (con SQL traite una window function). Poi fare uno shuffle e sort, ed infine applicare un reduce. In Fig(3.4) vediamo un esempio concreto di due funzioni in python. values in `reducer_count_ratings` è come se fosse una tupla dove vado a sommare ogni singolo componente.

Stiamo lavorando su una macchina virtuale chiamata Hortonworks Sandbox, che simula un cluster Hadoop per fare esercizi. Su Windows si usa PuTTY, che è un client SSH. Una volta connesso, sei l'utente `maria_dev`, cioè un utente normale senza privilegi amministrativi. Root è l'utente "super amministratore" su Linux (analogo del DBA in SQL). Può fare qualsiasi cosa sul sistema, ad esempio installare software, modificare repository, cambiare permessi di file. Per diventare root si usa: `su root`. Una volta diventato root, puoi modificare i repository e installare pacchetti come python-pip o nano. Sul nostro utente, `maria_dev`, scarichiamo i dati e lo script Python che implementa il job MapReduce.

```

cd /home/maria_dev
wget http://media.sundog-soft.com/hadoop/RatingsBreakdown.py
wget http://media.sundog-soft.com/hadoop/ml-100k/u.data

```

Ho poi eseguito il comando:

```

python RatingsBreakdown.py -r hadoop --hadoop-streaming-jar
/usr/hdp/current/hadoop-mapreduce-client/hadoop-streaming.jar u.data

```

dove:

- `python RatingsBreakdown.py` lancia lo script Python che hai scaricato (`RatingsBreakdown.py`). Questo script contiene il tuo job MapReduce scritto con MRJob.
- `-r hadoop` è un'opzione di MRJob che indica dove eseguire il job. Ovvero esegui il job sul cluster Hadoop presente nella sandbox (non in locale).
- `-hadoop-streaming-jar /usr...hadoop-streaming.jar` Hadoop MapReduce lavora usando lo streaming: i tuoi mapper e reducer (Python) comunicano con Hadoop tramite file temporanei e pipe. Questo parametro dice a MRJob dove trovare il jar di Hadoop streaming necessario per far girare Python sul cluster.

3.2 Esempio MapReduce con 3 nodi

Consideriamo il file seguente diviso riga per riga su 3 nodi:

- **Nodo 1:** "Ciao ciao amore, come stai?"
- **Nodo 2:** "Bene amore mio, tu come stai?"
- **Nodo 3:** "Tutto bene, mio cuore."

1. Fase Map

Ogni nodo genera coppie chiave-valore locali:

- Nodo 1:

```
("Ciao", 1), ("Ciao", 1), ("amore", 1), ("come", 1), ("stai", 1)
```

- Nodo 2:

```
("Bene", 1), ("amore", 1), ("mio", 1), ("tu", 1), ("come", 1), ("stai", 1)
```

- Nodo 3:

```
("Tutto", 1), ("bene", 1), ("mio", 1), ("cuore", 1)
```

2. Fase Shuffle & Sort

Hadoop raggruppa tutte le coppie per chiave e le invia ai nodi che eseguiranno il Reduce:

- Nodo 1: Ciao, stai, tu
- Nodo 2: amore, Bene, Tutto
- Nodo 3: come, mio, cuore

3. Fase Reduce

Ogni nodo aggrega i valori ricevuti:

- Nodo 1:

`("Ciao", 2), ("stai", 2), ("tu", 1)`

- Nodo 2:

`("amore", 2), ("Bene", 1), ("Tutto", 1)`

- Nodo 3:

`("come", 2), ("mio", 2), ("cuore", 1)`

Output finale

Hadoop scrive su HDFS il risultato aggregato:

```
"Ciao" → 2
"amore" → 2
"come" → 2
"stai" → 2
"Bene" → 1
"mio" → 2
"tu" → 1
"Tutto" → 1
"cuore" → 1
```

Chapter 4

PIG

Apache Pig è uno strumento che semplifica l'elaborazione di grandi moli di dati su Hadoop. Invece di scrivere manualmente le funzioni di *mapper* e *reducer*, come avviene con MapReduce in Java o in Python, Pig introduce un proprio linguaggio dichiarativo chiamato **Pig Latin**. Questo linguaggio consente di descrivere le trasformazioni e le analisi sui dati in maniera compatta e leggibile.

Dal punto di vista architetturale, la gerarchia può essere descritta come segue:

- Gli utenti scrivono gli script in Pig Latin.
- Gli script Pig vengono tradotti automaticamente in una sequenza di job MapReduce (oppure, in alternativa, in job eseguiti su Apache Tez, che è più veloce ed efficiente di MapReduce).
- I job generati da Pig vengono poi eseguiti su **YARN**, il gestore delle risorse del cluster Hadoop.
- I dati su cui lavorano questi job risiedono tipicamente in **HDFS** (Hadoop Distributed File System).

In altre parole, Pig si colloca *al di sopra* di MapReduce e di Tez, offrendo un livello di astrazione più alto: l'utente scrive semplici istruzioni Pig Latin e lascia che il sistema le traduca nei dettagli esecutivi (mapper, reducer, shuffle, sort). Questo rende Pig uno strumento particolarmente utile per analisi veloci e per prototipazione, poiché riduce drasticamente il tempo e la complessità di sviluppo rispetto alla scrittura manuale di codice MapReduce.

Quindi, per iniziare, scriveremo uno script di Pig che esamina l'insieme dei dati di movielens, trovando i film a cinque stelle più vecchi che può trovare, in modo da trovare i film vecchi più belli.

Anzitutto devo portare i dati su pig. Per farlo, basta fare: `ratings = LOAD '/user/maria_dev/ml-100k/u.data' AS (userID:int, movieID:int, rating:int, ratingTime:int);` dove il path è quello che porta all'HDFS. Un'altra differenza tra Pig e un database relazionale è che lo schema effettivo viene applicato quando si leggono i dati, senza che vi sia alcun tipo di struttura o schema o tipi associati. Siccome vogliamo sapere anche il nome del film, dobbiamo importare anche `u.item`. Ma ricordo che `u.item` non è separato dai tab ma da |, per cui: `metadata = LOAD '/user/maria_dev/ml-100k/u.item' USING PigStorage('|') AS (movieID:int, movieTitle:chararray, relaseDate: chararray, videoRelease:chararray, imdbLink:chararray);` Il

comando `DUMP metadata`; consente di scaricare il contenuto dell'intera relazione. Ricordiamo che vogliamo ordinare le cose in base alla data di uscita e per farlo dobbiamo inserire la data di uscita in un formato che possa essere ordinato. Grazie al comando: `nameLookup = FOREACH metadata GENERATE movieID, movieTitle, ToUnixTime(ToDate(releaseDate, 'dd-MM-yyyy')) AS releaseTime`. La prossima cosa da fare è individuare tutte le valutazioni associate a ogni singolo film, perché è necessario determinare in qualche modo la valutazione media di ogni film. Quindi possiamo dire che creeremo una nuova relazione chiamata `ratingsByMovie` che sarà in grado di raggruppare le valutazioni in base all'ID del film: `ratingsByMovie = GROUP ratings BY movieID`; Ora, per calcolare la media delle valutazioni possiamo fare una cosa del genere: `avgRatings = FOREACH ratingsByMovie GENERATE group AS movieID, AVG(ratings.rating) AS avgRating`; Ora tutto ciò che dobbiamo fare è dire di filtrare la media delle valutazioni per la media delle valutazioni è superiore a 4.0. `fiveStarMovies = FILTER avgRatings BY avgRating > 4.0`; Ora dovremmo fare una JOIN per capire il nome del film: `fiveStarsWithData = JOIN fiveStarMovies BY movieID, nameLookup BY movieID`; L'ultima cosa da fare è ordinare le cose in base alla data di uscita e avremo la nostra risposta: `oldestFiveStarMovies = ORDER fiveStarsWithData BY nameLookup::releaseTime`;

4.0.1 Comandi principali in PIG

- **LOAD** : carica un dataset dall'HDFS in Pig. Esempio: `data = LOAD '/user/maria/input.csv' USING PigStorage(',') AS (id:int, nome:chararray)`;
- **STORE** : salva un dataset dall'ambiente Pig su HDFS. Esempio: `STORE data INTO '/user/maria/output' USING PigStorage(',')`;
- **DUMP** : stampa i risultati a video. Esempio: `DUMP data`;
- **FILTER** : filtra righe in base a una condizione. Esempio: `filtered = FILTER data BY id > 10`;
- **GROUP** : raggruppa dati per una o più colonne. Esempio: `grouped = GROUP data BY id`;
- **FOREACH ... GENERATE** : applica trasformazioni riga per riga o aggregazioni. Esempio: `results = FOREACH grouped GENERATE group, COUNT(data)`;
- **ORDER** : ordina i dati in base a una o più colonne. Esempio: `sorted = ORDER data BY id DESC`;
- **RANK** : assegna un rank numerico alle righe in base a un ordinamento. Esempio: `ranked = RANK data BY id`;
- **JOIN** : unisce due dataset su una chiave comune. Esempio: `joined = JOIN data1 BY id, data2 BY id`;
- **UNION** : combina due dataset con stesse colonne. Esempio: `combined = UNION data1, data2`;
- **DISTINCT** : restituisce solo righe uniche. Esempio: `uniqueData = DISTINCT data`;
- **LIMIT** : limita il numero di righe restituite. Esempio: `sample = LIMIT data 10`;

Chapter 5

Spark

Apache Spark è un framework di elaborazione dati veloce e scalabile, progettato per gestire grandi volumi di dati in modo efficiente. A differenza di strumenti come Pig, Spark offre un'ampia flessibilità: è possibile scrivere veri e propri script utilizzando linguaggi di programmazione reali come **Java**, **Scala** o **Python**, e svolgere complesse operazioni di manipolazione, trasformazione e analisi dei dati. Ciò che distingue Spark è il suo ricco ecosistema, che permette di affrontare una vasta gamma di compiti avanzati, tra cui:

- **Machine Learning** per creare modelli predittivi su grandi dataset;
- **Data Mining** e analisi dei dati complessi;
- **Graph Processing** per elaborare reti e grafi complessi;
- **Data Streaming** per analizzare flussi di dati in tempo reale.

Proprio come accade in molte tecnologie basate su Hadoop, anche Spark segue lo stesso schema generale. In pratica, si ha un *programma driver*, ossia un piccolo script che controlla l'esecuzione del lavoro. Il driver comunica con un *gestore di cluster*. Per esempio, si può utilizzare YARN, ma non è obbligatorio: Spark può funzionare su Hadoop, ma non dipende necessariamente da esso. Infatti, Spark dispone di un proprio gestore di cluster integrato e può anche appoggiarsi ad altri sistemi, come *Mesos*. Qualunque sia la scelta, il gestore si occupa di distribuire il lavoro su un intero cluster di computer commodity. Questo è il punto centrale: suddividere i dati e processarli in parallelo. Ogni processo di esecuzione mantiene una propria cache e un insieme di compiti da svolgere. La *cache* è fondamentale per le prestazioni: a differenza delle soluzioni basate esclusivamente su disco, dove l'accesso avviene sempre e solo tramite HDFS, Spark cerca di conservare quanto più possibile in RAM. Questa caratteristica rappresenta una delle chiavi della sua velocità. Un altro aspetto decisivo è l'uso dei *grafi aciclici diretti* (DAG), di cui parleremo più avanti. Grazie a queste scelte progettuali, Spark risulta estremamente veloce: può essere fino a 100 volte più rapido di MapReduce quando lavora interamente in memoria e circa 10 volte più rapido anche quando è limitato dall'accesso al disco.

Spark è composto da diversi livelli o librerie. Ad esempio **Spark Streaming** o **Spark SQL** o **MLlib** per problemi di Machine Learning. Per iniziare con Spark va bene usare Python ma più avanti è molto più utile Scala.

5.1 Resilient Distributed Dataset (RDD)

Dal punto di vista della programmazione un RDD è solo un insieme di dati. Più precisamente un RDD è una collezione di dati suddivisa in partizioni che possono essere distribuite su più nodi del cluster. Una volta creato, un RDD non può essere modificato (**immutabile**). Si possono solo generare nuovi RDD applicando trasformazioni. Spark è in grado di ricostruire automaticamente i dati persi grazie al meccanismo di lineage (la “storia” delle trasformazioni applicate). Per questo si dice **resiliente**. In python, in particolare in PySpark, il metodo `parallelize()` serve a creare un RDD a partire da una collezione locale di Python (una lista, ad esempio). Chiaramente si possono fare delle operazioni sugli RDD ad esempio con `map()` che applica una funzione a ogni elemento dell’RDD. Oppure `filter()` mantiene solo gli elementi che soddisfano una condizione (ad esempio solo numeri pari).

```
rdd = sc.parallelize([1, 2, 3, 4])
rdd2 = rdd.map(lambda x: x * 2)
print(rdd2.collect()) # [2, 4, 6, 8]
```

`sc` è lo `SparkContext`, l’oggetto che rappresenta la connessione tra il tuo programma (il driver) e il cluster Spark. Tuttavia non verrà avviato nulla all’interno dello script del driver finché non verrà richiamata una di queste funzioni: `collect`, `count` ecc. Si cerca quindi di creare una catena di dipendenze all’interno dello script del driver e solo quando viene richiamata l’azione si cerca di capire quale sia il percorso più rapido attraverso le dipendenze. Ed è a quel punto che viene avviato il lavoro sul cluster.

5.1.1 Find the movie with the lowest average rating (with RDD’s)

```
from pyspark import SparkConf, SparkContext

# Funzione per creare un dizionario {movieID: movieTitle}
def loadMovieNames():
    movieNames = {}
    with open("ml-100k/u.item") as f:
        for line in f:
            fields = line.split('|')
            movieNames[int(fields[0])] = fields[1]
    return movieNames

# Converti ogni riga di u.data in (movieID, (rating, 1.0))
def parseInput(line):
    fields = line.split()
    return (int(fields[1]), (float(fields[2]), 1.0))

if __name__ == "__main__":
    # Crea lo SparkContext
    conf = SparkConf().setAppName("WorstMovies")
    sc = SparkContext(conf = conf)

    # Carica il dizionario movieID -> movieTitle
    movieNames = loadMovieNames()
```

```

# Carica il file u.data da HDFS
lines = sc.textFile("hdfs:///user/maria_dev/ml-100k/u.data")

# Converti in (movieID, (rating, 1.0))
movieRatings = lines.map(parseInput)

# Somma voti e conteggi per ciascun movieID
ratingTotalsAndCount = movieRatings.reduceByKey(
    lambda movie1, movie2: (movie1[0] + movie2[0], movie1[1] + movie2[1])
)

# Calcola la media dei voti
averageRatings = ratingTotalsAndCount.mapValues(
    lambda totalAndCount: totalAndCount[0] / totalAndCount[1]
)

# Ordina per voto medio
sortedMovies = averageRatings.sortBy(lambda x: x[1])

# Take the top 10 results
results = sortedMovies.take(10)

# Print them out:
for result in results:
    print(movieNames[result[0]], result[1])

```

Anzitutto definiamo una funzione che restituisce un *dizionario* Python, dove:

- la **chiave** è il `movieID` (l'identificativo del film),
- il **valore** è il nome del film (`fields[1]` nel file `u.item`).

Questo dizionario ci permetterà in seguito di convertire facilmente gli ID dei film nei loro nomi leggibili quando stamperemo i risultati finali. Successivamente definiamo la funzione `parseInput`, che prende in input una riga del file `u.data` e la converte in una *tupla di tuple*. Ad esempio, se il film con ID 210 ha ricevuto un voto di 3 stelle, la funzione restituirà:

(210, (3, 1))

Ricordo che mettiamo un 1 perchè l'idea è capire il film che è stato valutato il numero di volte minore. Non il più basso!!! In questo modo, ogni film sarà associato sia alla somma dei voti che al numero di voti, informazioni necessarie per calcolare la media successivamente.

All'interno del blocco `if`, creiamo lo **SparkContext**, che rappresenta l'ambiente Spark in cui gireranno tutte le operazioni distribuite. Prima di creare lo `SparkContext`, definiamo una configurazione:

```

conf = SparkConf().setAppName("WorstMovies")
sc = SparkContext(conf = conf)

```

- `SparkConf()` crea un oggetto di configurazione, dove possiamo impostare parametri per l'applicazione Spark.
- `setAppName("WorstMovies")` assegna un nome all'applicazione, utile per identificarla nell'interfaccia di Spark.
- `SparkContext(conf=conf)` crea l'oggetto principale (`sc`) attraverso cui possiamo:
 - creare RDD,
 - leggere file da HDFS o da file locali,
 - applicare trasformazioni e azioni sui dati distribuiti.

Dopo aver creato lo `SparkContext`, carichiamo il dizionario dei film con la funzione creata in precedenza `loadMovieNames()`, e il file `u.data` presente su HDFS:

```
movieNames = loadMovieNames()
lines = sc.textFile("hdfs:///user/maria_dev/ml-100k/u.data")
```

- `movieNames` contiene la mappatura da `movieID` a nome del film.
- `lines` è un RDD, ogni elemento è una riga del file `u.data`. Lo `SparkContext` si occupa di distribuire queste righe su più nodi per poterle elaborare in parallelo.

Ora, `lines` è un RDD cioè una collezione distribuita di righe del file `u.data`. `map()` è una trasformazione di Spark che prende in input una funzione (in questo caso `parseInput`), applica la funzione a ogni elemento dell'RDD e restituisce un nuovo RDD con i risultati. Quidni `movieRatings` è un RDD in cui ogni elemento ha la forma (`movieID`, (`rating`, `1.0`)).

`reduceByKey()` è una trasformazione di Spark che raggruppa gli elementi per chiave (`movieID` in questo caso), applica una funzione di riduzione ai valori associati a ciascuna chiave, restituisce un nuovo RDD con una tupla per ogni chiave, contenente il valore "*ridotto*". La funzione lambda somma i voti: `movie1[0] + movie2[0]` ed i conteggi di ogni film: `movie1[1] + movie2[1]`. Ovvero, dopo `reduceByKey()`, per ogni film abbiamo una tupla:

$$(\text{movieID}, (\text{sommaTotaleVoti}, \text{numeroTotaleVoti}))$$

Ad esempio, se il film 210 ha ricevuto voti (3,1) e (4,1), il risultato sarà: (210, (7, 2)). Quello che sto dicendo è che la funzione lambda dentro `reduceByKey()` prende due valori con la stessa chiave: `movie1` è il primo valore (`rating`, 1) per quella chiave (210) mentre `movie2` è il secondo valore (`rating`, 1) o il risultato parziale accumulato fino a quel punto.

A questo punto è molto semplice fare la media perché vado a prendere la somma totale dei voti e la divido per il numero di voti. Ovvero, avrò un qualcosa tipo: (210, 7/2). Infine ordino per la media: cioè medie più basse corrispondono a film con voti più bassi e/o tanti voti. vado a prendere solo i primi 10 risultati.

Quando si eseguono gli script python di Spark non si può usare solo python perché non funzionerà. Si usa piuttosto la shell: `spark-submit` che configura tutto l'ambiente Spark e che gira sul cluster.

5.1.2 Trasformazioni sugli RDD

Le trasformazioni sono operazioni che producono un nuovo RDD a partire da uno esistente. Sono **lazy**, cioè non vengono eseguite immediatamente, ma solo quando viene chiamata un'azione. Alcuni esempi:

- **map**: applica una funzione a ogni elemento (relazione 1:1), viene usata per mappare ogni riga del primo RDD con ogni riga del secondo RDD.
- **flatMap**: come **map**, ma può restituire più valori o nessuno.
- **filter**: seleziona solo gli elementi che soddisfano una condizione.
- **distinct**: restituisce solo valori unici.
- **sample**: estrae un campione casuale.
- operazioni insiemistiche: **union**, **intersection**, **subtract**, **cartesian**.

Esempio, usando programmazione funzionale:

```
rdd = sc.parallelize([1,2,3,4])
squaredRDD = rdd.map(lambda x: x*x)
# restituisce [1, 4, 9, 16]
```

5.1.3 Azioni sugli RDD

Le azioni avviano realmente l'esecuzione del job, producendo un risultato. Esempi:

- **collect()**: restituisce tutti i risultati al driver.
- **count()**: conta il numero di elementi.
- **countByValue()**: conta le occorrenze di ciascun valore.
- **take(n)**: restituisce i primi n elementi.
- **top(n)**: restituisce i n elementi più grandi.
- **reduce(func)**: riduce tutti gli elementi a un singolo valore usando una funzione.

5.1.4 Lazy Evaluation

Spark utilizza la **lazy evaluation**. Finché vengono applicate solo trasformazioni, non viene eseguito alcun calcolo. Quando invece viene chiamata un'azione, Spark:

1. costruisce un grafo di dipendenze (DAG) tra gli RDD,
2. ottimizza il percorso di calcolo,
3. distribuisce i task agli executor,
4. produce il risultato richiesto.

Quando chiamo un action quello che faccio è dire a Spark: questo è il risultato che voglio ottenere, trova il modo più veloce per arrivarci. Questo approccio consente di ottimizzare le prestazioni ed evitare calcoli inutili.

5.2 Spark SQL

SparkSQL è un modulo di Apache Spark che permette di usare SQL per interrogare dati distribuiti. Consente di trattare dati strutturati (come CSV, JSON, Parquet o tabelle Hive) in modo simile a un database relazionale, ma su un cluster Spark. SparkSQL sfrutta Catalyst Optimizer, un motore di ottimizzazione delle query, e può convertire le query SQL in RDD distribuiti per l'esecuzione parallela.

Vedremo in un esempio che questa volta non importeremo più `SparkContext` ma `SparkSession` che comprende sia un contesto Spark che un contesto SQL. Importeremo anche l'oggetto `row`, che ci consentirà di creare cornici di dati di oggetti ed alcune funzioni SQL, in modo da poter fare cose come calcolare la media delle valutazioni man mano che procediamo.

```
from pyspark.sql import SparkSession
from pyspark.sql import Row
from pyspark.sql import functions

# Funzione per creare il dizionario movieID -> nome del film
def loadMovieNames():
    movieNames = {}
    with open("ml-100k/u.item") as f:
        for line in f:
            fields = line.split('|')
            movieNames[int(fields[0])] = fields[1]
    return movieNames

# Funzione che converte ogni riga in un oggetto Row(movieID, rating)
def parseInput(line):
    fields = line.split()
    return Row(movieID=int(fields[1]), rating=float(fields[2]))

if __name__ == "__main__":
    # Creazione della SparkSession (necessaria su Windows)
    spark = SparkSession.builder.appName("PopularMovies").getOrCreate()

    # Carica il dizionario movieID -> nome del film
    movieNames = loadMovieNames()

    # Legge il file u.data da HDFS
    lines = spark.sparkContext.textFile("hdfs:///user/maria_dev/ml-100k/u.data")

    # Converte le righe in RDD di Row(movieID, rating)
    movies = lines.map(parseInput)

    # Converte l'RDD in un DataFrame
    movieDataset = spark.createDataFrame(movies)

    # Calcola la media dei voti per ogni movieID
    averageRatings = movieDataset.groupBy("movieID").avg("rating")
```

```

# Calcola il numero di voti per ogni movieID
counts = movieDataset.groupBy("movieID").count()

# Fa il join tra count e media dei voti
averagesAndCounts = counts.join(averageRatings, "movieID")

# Estrae i primi 10 film ordinati per media dei voti
topTen = averagesAndCounts.orderBy("avg(rating)").take(10)

# Stampa i risultati convertendo gli ID in nomi dei film
for movie in topTen:
    print(movieNames[movie[0]], movie[1], movie[2])

# Chiude la sessione Spark
spark.stop()

```

Le funzioni `loadMovieNames` ed `parseInput` sono rimaste le stesse. Notiamo come non viene creato più uno *SparkContext* ma uno *SparkSession* che è il punto d'ingresso principale per lavorare con *DataFrame* e *SparkSQL*. Tutte le operazioni sui *DataFrame* passano attraverso la *SparkSession*. *builder* è un costruttore di sessione: serve per configurare parametri della sessione Spark prima di crearla, ad esempio: nome dell'applicazione, configurazioni di memoria, modalità locale o cluster. `.appName("PopularMovies")` Imposta il nome dell'applicazione Spark. Questo nome apparirà nell'interfaccia web di Spark, utile per monitorare i job, le fasi e le statistiche. Infine, `.getOrCreate()` crea una *SparkSession* con le configurazioni impostate. Se esiste già una sessione attiva, la riutilizza invece di crearne una nuova e quindi garantisce che ci sia sempre una sola *SparkSession* attiva in quel programma. Un **DataFrame** in Spark è una struttura dati tabellare, simile a una tabella di database o a un foglio di calcolo. L'idea è trasformare un RDD in un *DataFrame* e per farlo `createDataFrame(movies)` converte quell'RDD in un *DataFrame* con colonne: `movieID` (int), `rating` (float). Da qui il gioco è molto più semplice perché si possono fare operazioni simili a un database relazionale:

- **SELECT** → tramite `select()` sul *DataFrame* oppure `spark.sql("SELECT ...")`
- **WHERE** → tramite `filter()` sul *DataFrame* oppure clausola `WHERE` in SQL
- **GROUP BY** → tramite `groupBy()` sul *DataFrame*
- **JOIN** → per unire più *DataFrame* basati su una o più colonne chiave
- **ORDER BY** → per ordinare le righe, tramite `orderBy()` sul *DataFrame* o `ORDER BY` in SQL

Il resto del codice fa esattamente come lo script visto in precedenza.

Ora, bisogna dirgli esplicitamente se si vuole usare Spark 2 per farlo: `SPARK_MAJOR_VERSION=2` questo dirà a Hortonworks che vogliamo utilizzare Spark 2 invece di Spark

5.3 Spark using MLlib

L'idea qui è quella di consigliare all'utente dei film in base a quello che piace all'utente, ed in base a cosa ha visto. L'idea è la stessa su cui si basa Netflix, oppure Amazon con i prodotti consigliati.

Anzitutto ho creato un personaggio nel file `u.data` con `userID = 0`. Per semplicità lo chiameremo Mario, ma ricordiamoci che è l'utente zero. Mario ha valutato 3 film di cui a due di essi ha dato 5 stelle: *Guerre stellari* e *L'impero colpisce ancora*; mentre ha dato una stella a *Via col vento*. Quindi, date queste tre valutazioni, chiederemo al modello ALS di prevedere le valutazioni di quell'utente per ogni altro film. L'idea generale quindi è quella di anzitutto cercare l'utente zero. Ora prevediamo le nostre valutazioni per altri film: per evitare risultati errati, limiterò l'analisi ai film che sono stati valutati più di 100 volte. Quindi, le cose per le quali abbiamo una quantità ragionevole di dati sulla valutazione delle persone e sugli elementi con cui le hanno valutate.

```
from pyspark.sql import SparkSession
from pyspark.ml.recommendation import ALS
from pyspark.sql import Row
from pyspark.sql.functions import lit

# Carica il dizionario movieID -> nome del film
def loadMovieNames():
    movieNames = {}
    with open("ml-100k/u.item") as f:
        for line in f:
            fields = line.split('|')
            movieNames[int(fields[0])] = fields[1].decode('ascii',
                'ignore')
    return movieNames

# Converti ogni riga del file u.data in Row(userID, movieID, rating)
def parseInput(line):
    fields = line.value.split()
    return Row(userID=int(fields[0]), movieID=int(fields[1]), rating=
        float(fields[2]))

if __name__ == "__main__":
    # Creazione della SparkSession
    spark = SparkSession.builder.appName("MovieRecs").getOrCreate()

    # Necessario su HDP 2.6.5 per join incroci
    spark.conf.set("spark.sql.crossJoin.enabled", "true")

    # Carica il dizionario movieID -> nome del film
    movieNames = loadMovieNames()

    # Legge i dati da HDFS e li converte in RDD
    lines = spark.read.text("hdfs:///user/maria_dev/ml-100k/u.data").rdd

    # Converti in RDD di Row(userID, movieID, rating)
    ratingsRDD = lines.map(parseInput)

    # Converti in DataFrame e mette in cache
    ratings = spark.createDataFrame(ratingsRDD).cache()
```

```

# Crea il modello ALS (Collaborative Filtering) con tutto il dataset
als = ALS(maxIter=5, regParam=0.01, userCol="userID",
itemCol="movieID", ratingCol="rating")
model = als.fit(ratings)

# Stampa i voti dell'utente 0
print("\nRatings for user ID 0:")
userRatings = ratings.filter("userID = 0")
for rating in userRatings.collect():
    print(movieNames[rating['movieID']], rating['rating'])

print("\nTop 20 recommendations:")
# Seleziona i film valutati piu di 100 volte
ratingCounts = ratings.groupBy("movieID").count()
.filter("count > 100")

# Crea un DataFrame di test per l'utente 0 con tutti i film popolari
popularMovies = ratingCounts.select("movieID")
.withColumn('userID', lit(0))

# Genera raccomandazioni per l'utente 0
recommendations = model.transform(popularMovies)

# Prende i top 20 film con rating previsto piu alto
topRecommendations = recommendations.sort
(recommendations.prediction.desc()).take(20)

for recommendation in topRecommendations:
    print(movieNames[recommendation['movieID']],
recommendation['prediction'])

# Chiude la sessione Spark
spark.stop()

```

Volendo spiegazione...

5.3.1 Esercizio

Se ci si pensa i nostri risultati nei nostri esercizi in cui stavamo guardando i film con i voti peggiori non erano molto significativi perché abbiamo ottenuto molti risultati che sono stati valutati da una sola persona. Ma il fatto che una sola persona dà a un film una valutazione di una stella, è davvero giusto definirlo un brutto film?

Quindi, basandoci sull’RDD (il primo esempio visto insieme) modifichiamo lo script in modo tale che il film è stato votato più di 10 volte. Per farlo ho aggiunto dopo `reduceByKey()` la seguente riga di codice: `ratingTotalsAndCount10 = ratingTotalsAndCount.filter(lambda x: x[1][1] > 10)` dove `x[1][1]` è il secondo elemento della tupla interna di `ratingTotalsAndCount`. Ovvero se abbiamo (210, (7, 5)) io controllo se 5 > 10.

Chapter 6

Hive

Apache Hive è un sistema di **data warehousing** costruito sopra l'ecosistema Hadoop. Non si tratta di un database relazionale tradizionale, bensì di uno strumento che permette di interrogare grandi quantità di dati memorizzati in *HDFS* attraverso un linguaggio simile a SQL, chiamato *HiveQL* (o *HQL*).

Le query espresse in HiveQL non vengono eseguite direttamente, ma tradotte in job di basso livello (inizialmente *MapReduce*, oggi più frequentemente *Tez* o *Spark*), che sfruttano l'elaborazione distribuita per garantire scalabilità ed efficienza su cluster di grandi dimensioni. Hive adotta un approccio *schema-on-read*, applicando lo schema ai dati solo al momento della lettura; ciò consente di lavorare con dati strutturati.

Hive è **scalabile**, ovvero il può funzionare con dati distribuiti su un intero cluster di computer, a differenza di un database tradizionale, dove siamo limitati su un singolo host. Si parla quindi di sistemi OLAP, Hive non è usato per sistemi OLTP. Poi c'è HiveQL che fondamentalmente è come SQL. Tuttavia ci sono delle differenze, una di queste sono le viste. Le viste in HiveQL sono di fatto delle tabelle ma con una logica. Non sono come le viste materializzate, perché quest'ultime sono una copia dei dati ed immagazzinati. Nelle viste di Hive invece c'è una logica dietro ma di fatto non è una banale copia dei dati.

6.1 Caricamento file su Hive

per caricare i file (tabelle) su Hive bisogna andare su Hive view e poi schiacciare *upload table*, come si vede in Fig (6.1). Osserviamo come di fatto il **GROUP BY** è un'operazione di riduzione,

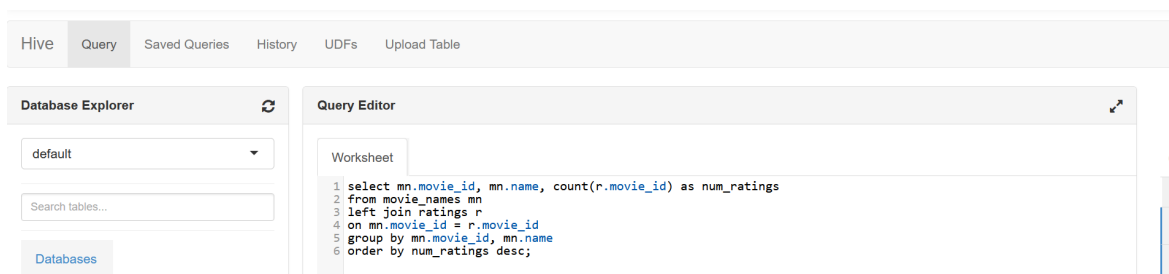


Figure 6.1: Hive world.

ed è la magia di Hive che traduce effettivamente il comando SQL in mapper e reducer. Siccome siamo in un sistema OLAP, fare JOIN è abbastanza doloroso, visto la ridondanza.

6.2 Schema On Read - Hive

Uno dei concetti base di Hive è qualcosa chiamato *Schema On Read* ed è questo ciò che lo separa da un database più tradizionale.

- **DB-Relazionale:** Un database relazionale utilizza *Schema on Write*. È il modello classico (es. Oracle, MySQL, PostgreSQL). Di fatto cosa succede: quando carichi i dati in tabella, il DBMS controlla subito che rispettino lo schema (tipi di dato, vincoli, chiavi, ecc.). Se i dati non rispettano le regole allora il caricamento fallisce. Ad esempio, se creo una tabella utenti con id (int), nome (varchar) ed età (int) e poi ci inserisco come età 'venti' il sistema dà errore. E' da qui che nasce l'ETL.
- **Hive:** si basa sullo schema on read ovvero i dati vengono caricati "così come sono" in HDFS, senza alcun controllo. Lo schema viene applicato solo al momento della lettura (quando fai una query). Questo dà molta più libertà perché i file su HDFS possono essere CSV, JSON, Parquet, ecc.

Chiaramente ci sono pregi e difetti da entrambi i lati ed a seconda di quello che hai bisogno usi uno o l'altro. Ad esempio uno svantaggio di usare Hive è che le query possono essere più lente perché Hive deve interpretare lo schema al volo. Inoltre se i dati contengono errori o inconsistenze lo scopri solo al momento della query.

6.2.1 LOAD DATA vs LOAD DATA LOCAL

Quando stiamo importando dei dati tramite HiveQL, ad esempio:

```
LOAD DATA INPATH '/home/maria_dev/u.data'
INTO TABLE ratings;
```

Qui il file **u.data** si trova già su **HDFS** (file system distribuito). Quando esegui il comando: Hive non copia i dati → li sposta nella cartella interna della tabella (/warehouse/ratings/ ad esempio). Dopo il comando, il file non si trova più nella posizione originale. In pratica Hive diventa proprietario di quel file. Di conseguenza, se fai **DROP TABLE ratings**, anche i dati su HDFS vengono eliminati. Mentre:

```
LOAD DATA LOCAL INPATH '/home/maria_dev/u.data'
INTO TABLE ratings;
```

Qui il file **u.data** si trova nel file system locale (ad esempio sul nodo edge o sulla tua macchina). Quando esegui il comando Hive copia il file dal tuo file system locale dentro HDFS, nella cartella interna della tabella. I dati originali rimangono intatti nella tua directory locale.

6.2.2 Managed Tables vs External Tables

Per la Managed Tables la sintassi tipica è:

```
CREATE TABLE ratings (
  user_id INT,
  movie_id INT,
  rating INT,
  rating_time STRING
```

```
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY '\t'  
STORED AS TEXTFILE;
```

I dati vengono spostati/copiati dentro il “warehouse” di Hive, per cui Hive diventa proprietario dei dati. Se fai DROP TABLE ratings Hive elimina sia lo schema sia i dati fisici su HDFS. Mentre con le External Tables (Tabelle Esterne):

```
CREATE EXTERNAL TABLE ratings (  
  user_id INT,  
  movie_id INT,  
  rating INT,  
  rating_time STRING  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY '\t'  
STORED AS TEXTFILE  
LOCATION '/user/maria_dev/ml-100k/';
```

I dati rimangono nella loro posizione originale su HDFS (/user/maria_dev/ml-100k/). Hive non ne ha la “proprietà”, semplicemente li interpreta con lo schema che hai definito.

6.2.3 Partizionamento

Un altro argomento importante con Hive è il partizionamento. Come sappiamo il partizionamento è un modo per ottimizzare le query su grandi dataset. Invece di cercare in tutti i file della tabella, Hive può limitare la scansione solo ai file di una partizione specifica. In Hive la colonna di partizionamento non è una colonna normale dentro i file: Hive la usa solo per creare sottodirectory nel file system. Ad esempio la tabella clienti partizionata per country, vuol dire:

```
/user/hive/warehouse/clienti/country=CA/  
/user/hive/warehouse/clienti/country=US/  
/user/hive/warehouse/clienti/country=FR/
```

Quando fai una query su una partizione specifica, Hive legge solo la cartella corrispondente, risparmiando tempo.

6.2.4 Esercizio

Trovare i film con il punteggio medio più alto nel dataset **MovieLens**, considerando solo i film votati più di 10 volte. In questo modo si evitano risultati distorti da film con pochissimi voti, ad esempio film a 5 stelle valutati da una sola persona.

```
SELECT mn.movie_id, mn.name, AVG(r.rating) AS MeanRating  
FROM movie_names mn  
JOIN ratings r  
  ON mn.movie_id = r.movie_id  
GROUP BY mn.movie_id, mn.name
```

```
HAVING COUNT(r.rating) > 10  
ORDER BY MeanRating DESC;
```

6.3 Integrating MySQL and Hadoop (Sqoop)

Sqoop è uno strumento progettato per il trasferimento di dati tra sistemi relazionali e l'ecosistema Hadoop. Permette di importare/esportare dati tra database come MySQL e Hadoop Distributed File System (HDFS), Hive o HBase, sfruttando la potenza di calcolo distribuita.

Sqoop sfrutta job di tipo *MapReduce* per il trasferimento: in particolare, vengono utilizzati soltanto i *mapper*, poiché l'operazione non richiede una fase di *reduce*.

Per preparare il database MySQL con i dati MovieLens, i passi principali sono stati i seguenti:

1. Creare il database `movielens`:

```
CREATE DATABASE movielens;
```

2. Importare il file SQL nella nostra istanza MySQL (utente `maria_dev`):

```
mysql -u maria_dev -p
```

3. Impostare correttamente la codifica dei caratteri:

```
SET NAMES 'utf8';  
SET CHARACTER SET 'utf8';
```

4. Selezionare il database MovieLens e caricare lo script SQL:

```
USE movielens;  
SOURCE movielens.sql;
```

5. Controllare che le tabelle siano state create correttamente:

```
SHOW TABLES;
```

6. Visualizzare alcuni dati per verificarne il contenuto:

```
SELECT * FROM movies LIMIT 10;
```

A questo punto è possibile eseguire le query SQL su MySQL per esplorare i dati, prima di importarli in Hadoop tramite Sqoop (vedi Fig. 6.2).

Qui però le query sono davvero estremamente più veloci.

```

Query OK, 2891 rows affected (0.08 sec)
Records: 2891 Duplicates: 0 Warnings: 0

Query OK, 0 rows affected (0.00 sec)

mysql> show tables;
+-----+
| Tables_in_movielens |
+-----+
| genres               |
| genres_movies        |
| movies               |
| occupations          |
| ratings              |
| users                |
+-----+
6 rows in set (0.02 sec)

mysql> select * from movies limit 10;
+----+-----+-----+
| id | title                | release_date |
+----+-----+-----+
| 1  | Toy Story (1995)     | 1995-01-01   |
| 2  | GoldenEye (1995)     | 1995-01-01   |

```

Figure 6.2: MySQL

6.4 Use Sqoop to import data from MySQL to HDFS/Hive

Per poter utilizzare Sqoop per importare dati da MySQL, è necessario prima assicurarsi che l'utente MySQL disponga delle autorizzazioni appropriate. Nel nostro esempio, l'utente `root` deve poter accedere al database `movielens` e interagire con le sue tabelle. Questo viene ottenuto tramite il comando SQL:

```
GRANT ALL PRIVILEGES ON movielens.* TO root@localhost IDENTIFIED BY 'hadoop';
```

Una volta configurate le autorizzazioni, è possibile avviare l'importazione dei dati da MySQL verso Hadoop utilizzando Sqoop. Il comando utilizzato è il seguente:

```
sqoop import \
--connect jdbc:mysql://localhost/movielens \
--driver com.mysql.jdbc.Driver \
--table movies \
-m 1 \
--username root \
--password hadoop
```

In questo esempio, l'importazione avviene con un solo mapper (`-m 1`), il che significa che i dati vengono trasferiti in modo sequenziale. In un cluster reale, sarebbe possibile sfruttare la parallelizzazione, lasciando attivi più mapper, in modo da rendere l'importazione più veloce e scalabile. **JDBC** sta per Java Database Connectivity. È un'API di Java che permette a programmi scritti in Java di interagire con database relazionali. In pratica, JDBC è il “ponte”

tra il tuo programma Java (o strumenti basati su Java, come Sqoop) e un database come MySQL, PostgreSQL, Oracle, ecc.

Dopo aver eseguito questo comando, i dati della tabella `movies` sono disponibili in Hadoop. Tuttavia, è anche possibile importare direttamente i dati in Hive, evitando il passaggio intermedio tramite HDFS. Questo si ottiene aggiungendo l'opzione `-hive-import` al comando Sqoop:

```
sqoop import \  
--connect jdbc:mysql://localhost/movielens \  
--driver com.mysql.jdbc.Driver \  
--table movies \  
-m 1 \  
--username root \  
--password hadoop \  
--hive-import
```

Con questa opzione, i dati vengono caricati direttamente in una tabella Hive corrispondente, rendendoli immediatamente disponibili per le query SQL all'interno dell'ambiente Hive.

6.5 Use Sqoop to import data from Hive to MySQL

Prima di poter esportare i dati da Hive verso MySQL, è importante capire dove risiedono effettivamente i dati all'interno di Hive. Navigando nella visualizzazione dei file HDFS, possiamo osservare la posizione fisica dei dati. Su Hortonworks, i dati Hive risiedono tipicamente nella directory delle applicazioni, sotto Hive, nella cartella `warehouse`. Per la nostra tabella `movies`, i dati si trovano nel percorso:

```
/apps/hive/warehouse/movies/part-m-00000
```

Prima di eseguire l'esportazione dei dati in un database MySQL tramite Sqoop, è necessario assicurarsi che la tabella di destinazione esista già e sia pronta a ricevere i dati. Nel nostro esempio, creiamo la tabella `exported_movies` con il seguente comando SQL:

```
CREATE TABLE exported_movies (  
    id INTEGER,  
    title VARCHAR(255),  
    releaseData DATE  
);
```

Con la tabella pronta, possiamo procedere all'esportazione dei dati da Hive a MySQL utilizzando Sqoop. Il comando utilizzato è il seguente:

```
sqoop export \  
--connect jdbc:mysql://localhost/movielens \  
-m 1 \  
--driver com.mysql.jdbc.Driver \  
--table exported_movies \  
--export-dir /apps/hive/warehouse/movies \  

```



```
--input-fields-terminated-by '\001' \  
--username root \  
--password hadoop
```

In questo comando, l'opzione `-export-dir` indica il percorso dei dati Hive da esportare, mentre `-input-fields-terminated-by '\001'` specifica il delimitatore utilizzato nei file generati da Hive. L'opzione `-m 1` utilizza un solo mapper per l'esportazione, ma in un ambiente di produzione si possono utilizzare più mapper per parallelizzare il trasferimento dei dati.

Chapter 7

DataBase NON-Relazionali con Hadoop

L'integrazione di Hadoop con soluzioni SQL, come MySQL e altri sistemi di gestione di database relazionali (RDBMS), permette di sfruttare la potenza di un linguaggio di query analitico come SQL per rispondere a domande complesse sui dati. I database relazionali richiedono tempi di esecuzione significativi; risultano quindi appropriati per analisi su dataset di dimensioni contenute o per applicazioni interne con volumi di dati limitati.

Tuttavia, in presenza di dataset di grandi dimensioni e di requisiti di risposta rapida a query specifiche, i database relazionali possono presentare limitazioni. Ad esempio, potrebbero risultare insufficienti per rispondere a interrogazioni ad altissima frequenza su dataset di vasta scala, come quelle necessarie per sistemi web di grandi dimensioni (Amazon, Google). I database NoSQL costituiscono un'alternativa ai sistemi relazionali. Essi rinunciano a un linguaggio di query completo come SQL in favore della capacità di rispondere rapidamente a domande semplici su dataset di grandi dimensioni, ad esempio: "*Quali prodotti sono stati ordinati da questo cliente?*" o "*Quali pagine web ha visitato questo utente?*".

I sistemi NoSQL sono progettati per scalare orizzontalmente, garantendo velocità e resilienza. HBase, costruito su HDFS, rappresenta un esempio di database NoSQL transazionale e scalabile, in grado di esporre dati su cluster Hadoop a servizi web o applicazioni che richiedono accessi rapidi e a elevata scala. In questi contesti, è spesso sufficiente disporre di un'interfaccia chiave-valore (key-value), che consenta di ottenere rapidamente informazioni associate a un identificatore specifico, senza la necessità di un linguaggio di query completo.

7.1 API

Nei database relazionali (RDBMS), il linguaggio standard per interrogare e modificare i dati è SQL. Il client invia query SQL al database. Nei database non relazionali, spesso non esiste un linguaggio unico come SQL. Al contrario, l'accesso avviene tramite API specifiche del database (ad esempio, MongoDB offre un'API con metodi come `find()`, `insertOne()`, `updateOne()`).

Le API sono il modo principale con cui si interagisce con i database non relazionali (NoSQL). Un'API (Application Programming Interface) è un insieme di regole e strumenti che permette a due software di comunicare tra loro. In pratica, definisce come un programma può “chiedere” servizi o dati a un altro programma senza preoccuparsi di come questi siano implementati internamente.

Le API dei database NoSQL sono quasi sempre progettate attorno alle operazioni **CRUD**. CRUD sta per Create, Read, Update, Delete le quattro operazioni fondamentali su qualsiasi dato.

7.1.1 HBase

HBase è un database NoSQL distribuito basato su colonne, costruito sopra HDFS. Per fare chiarezza:

- HBase è il sistema che archivia e gestisce i dati su un cluster Hadoop.
- Fornisce un'API (Java, REST, Thrift, ecc.) per accedere ai dati: quindi interagisci con HBase tramite l'API, ma HBase stesso è il database.

7.1.2 Architettura di HBase - Partizionamento orizzontale

L'architettura HBase ha 3 componenti principali: HMaster, Region Server, Zookeeper.

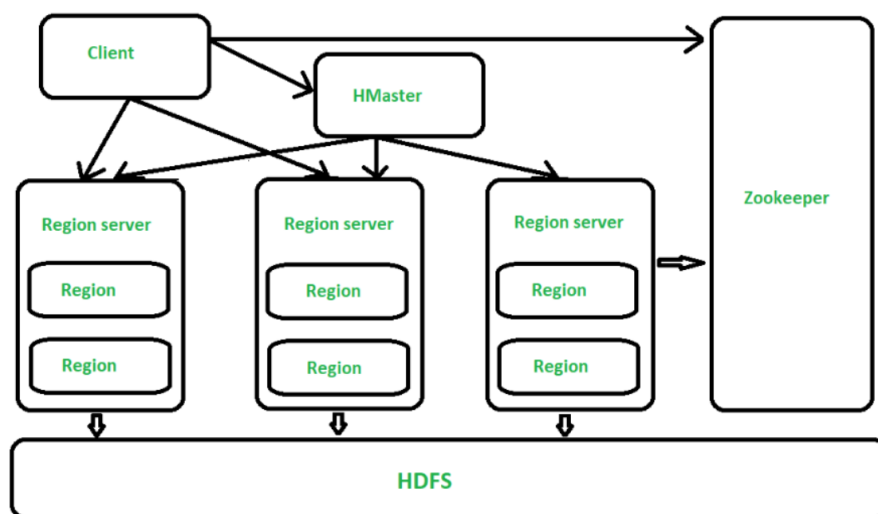


Figure 7.1: Architettura di HBase

HBase è un database distribuito costruito su HDFS, progettato per gestire grandi quantità di dati in maniera scalabile e resiliente. I dati sono organizzati in tabelle, le quali sono suddivise in regioni. Ogni regione rappresenta un intervallo continuo di righe ed è l'unità fondamentale di distribuzione dei dati all'interno del cluster. Le regioni sono gestite dai RegionServer, che si occupano delle operazioni di lettura e scrittura sui dati. L'HMaster coordina il cluster, assegnando le regioni ai RegionServer e gestendo operazioni di alto livello come la creazione o la cancellazione delle tabelle. I dati all'interno delle tabelle sono strutturati in famiglie di colonne, ciascuna delle quali contiene le colonne effettive, permettendo di raggruppare e memorizzare insieme i dati correlati per ottimizzare le prestazioni. Infine, Zookeeper è un servizio esterno che HBase utilizza per mantenere la sincronizzazione del cluster, la gestione delle configurazioni e il monitoraggio dello stato dei nodi.

- Una tabella HBase è divisa in righe identificate da una **row key univoca**.
- Ogni riga può avere più famiglie di colonne (column families).
- Ogni famiglia di colonne contiene colonne multiple:

- I dati sono memorizzati per famiglia di colonne, quindi tutte le colonne di una stessa famiglia sono memorizzate insieme sul disco, il che migliora le prestazioni di lettura/scrittura.
- Una regione è un intervallo di righe della tabella: contiene tante righe, ognuna con le sue famiglie di colonne e le colonne effettive.

Esempio di struttura base di una tabella HBase

Consideriamo una tabella chiamata **Users** in HBase.

Row Key	Personal		Contact	
	Name	Age	Email	Phone
user1	Alice	30	alice@example.com	1234567890
user2	Bob	25	bob@example.com	0987654321
user3	Carol	28	carol@example.com	1122334455

- **Row Key:** Identificatore univoco della riga (ad esempio **user1**, **user2**, ...).
- **Famiglia di colonne:** Raggruppamento logico di colonne, memorizzate insieme sul disco. Nell'esempio, **Personal** e **Contact**.
- **Colonne:** Contengono i dati effettivi all'interno di ciascuna famiglia di colonne. Ad esempio, nella famiglia **Personal** abbiamo **Name** e **Age**.
- **Righe:** Ogni riga ha una row key e può avere valori per tutte o alcune colonne delle famiglie.
- **Regione:** Un intervallo di righe della tabella; un **RegionServer** gestisce una o più regioni.

La tabella è divisa in **regioni**, ogni regione contiene un insieme di **righe**, e ogni riga può avere più **famiglie di colonne** (column families). Ogni famiglia di colonne contiene una o più **colonne** con i dati effettivi. Questo è quello che si chiama un **partizionamento orizzontale** applicato a livello di cluster distribuito.

Questo tipo di struttura permette di gestire grandi quantità di dati distribuiti in maniera scalabile e veloce.

7.2 Import data in HBase

Vogliamo creare una tabella HBase per le classificazioni dei film raggruppate per utente. Ovvero per un dato ID utente, restituiscimi tutti i voti per quell'ID utente. Quindi lo schema qui è l'ID di riga ha una chiave univoca di "UserID", e ogni ID utente ha una famiglia di colonne "Rating" e questa famiglia di colonne "Rating" può essere composta da un numero qualsiasi di singoli film. per ID film 50 era 1 stella, la valutazione di questo utente per ID film 33 era 5 stelle, la valutazione di questo utente per ID film 223 era 5

7.3 Cassandra

Cassandra è un database distribuito progettato con l'obiettivo primario della massima disponibilità dei dati. A differenza di altri sistemi, non prevede un nodo centrale: tutti i nodi del cluster sono equivalenti e cooperano tra loro, eliminando così il rischio di un singolo punto di guasto.

Il modello dei dati si ispira a quello di BigTable e HBase, basato su famiglie di colonne organizzate in modo da garantire un'alta efficienza nelle letture e scritture distribuite. Per l'interazione con i dati Cassandra utilizza il linguaggio CQL (Cassandra Query Language), che adotta una sintassi simile a SQL, ma con funzionalità più limitate in termini di join e operazioni complesse.

La scelta di Cassandra, rispetto ad altri database non relazionali, è motivata soprattutto dalla sua capacità di scalare orizzontalmente senza interruzioni e di mantenere elevata la disponibilità anche in presenza di guasti. Questo lo rende particolarmente adatto a scenari in cui la continuità del servizio e la distribuzione geografica dei dati sono più importanti della ricchezza del linguaggio di query o della rigidità delle garanzie transazionali.

7.3.1 Il Teorema di CAP

In HDFS i file vengono spezzati in blocchi (es. 128 MB ciascuno) e distribuiti sui nodi del cluster. Questa operazione non la fa un programma generico, ma il **NameNode** che è il nodo “master”: in Hadoop c'è un main node (il NameNode) che gestisce il tutto, mentre i DataNode sono “esecutori” che conservano i dati. HBase gira su HDFS quindi è basato su un database clusterizzato con in NameNode.

Cassandra è un database distribuito che elimina il concetto di nodo centrale, garantendo così l'assenza di un singolo punto di guasto e privilegiando la disponibilità dei dati. Il suo modello si ispira a quello di BigTable e HBase, con famiglie di colonne come struttura di base, ed è accessibile tramite il linguaggio CQL, simile a SQL ma meno espressivo.

La peculiarità di Cassandra emerge chiaramente se lo si osserva attraverso il teorema di CAP, secondo cui un sistema distribuito non può garantire contemporaneamente coerenza (consistency), disponibilità (availability) e tolleranza delle partizioni (partition tolerance).

- **Coerenza:** Un sistema è consistente se, quando scrivi o aggiorni un dato, tutti i nodi del cluster vedono subito lo stesso valore. Ad esempio, in un database bancario, se trasferisci 100€ da un conto a un altro, vuoi che su qualsiasi nodo tu legga, il saldo sia aggiornato in modo coerente.
- **Availability:** Un sistema è disponibile se ogni richiesta ottiene sempre una risposta. Ovvero, su un sito di e-commerce, anche se un nodo non ha l'ultimo aggiornamento dello stock, deve comunque rispondere “ci sono 5 prodotti” invece di bloccarsi.
- **Partition Tolerance:** Un sistema è tollerante alle partizioni se continua a funzionare anche in caso di guasti nella rete

Cassandra, ad esempio, sceglie Availability + Partition Tolerance (AP): privilegia la disponibilità, accettando che la coerenza sia eventuale (eventual consistency).

HBase o sistemi simili tendono più verso Consistency + Partition Tolerance (CP).

Nel caso di Cassandra, la progettazione è orientata alla disponibilità e alla tolleranza delle partizioni, mentre la coerenza viene sacrificata a favore della continuità del servizio. In altre parole,

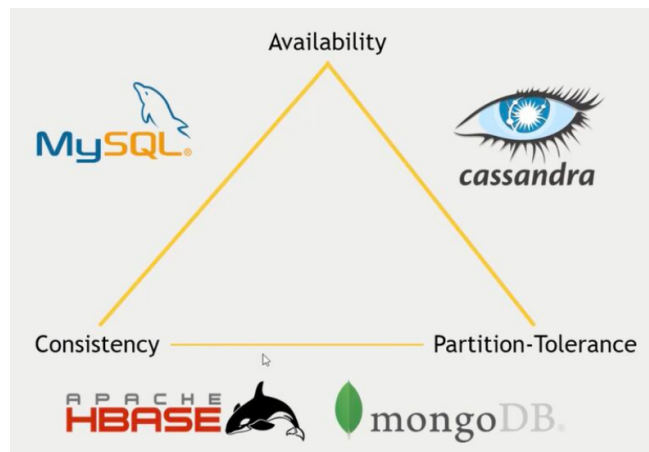


Figure 7.2: Collocazione nel triangolo CAP

anche in presenza di guasti o di partizioni di rete, il sistema continua a rispondere alle richieste, accettando che i dati possano temporaneamente risultare incoerenti tra i nodi.

Questa scelta architetturale rende Cassandra particolarmente adatto a contesti in cui è fondamentale non interrompere mai l'accesso ai dati, ad esempio in applicazioni globali distribuite o in sistemi che gestiscono grandi volumi di scritture in tempo reale.

7.3.2 Partizionamento Orizzontale - Cassandra (partition key)

In Cassandra non c'è un nodo principale: tutti i nodi sono equivalenti, quindi non c'è bisogno di master/region server come in HBase. Le righe di una tabella vengono distribuite tra i nodi del cluster usando una chiave di partizione. La chiave di partizione determina su quale nodo ogni riga verrà memorizzata.

Supponiamo di avere una tabella `users` con i campi: (`user_id`, `name`, `age`, `city`). Se vogliamo distribuire i dati tra i nodi del cluster, scegliamo una `partition key`. Ad esempio, usiamo `user_id` come `partition key`. Cassandra calcola un **hash**¹ della `partition key` (`user_id`) e decide su quale nodo memorizzare quella riga. Se aggiungiamo nodi al cluster, Cassandra può ridistribuire le righe calcolando di nuovo l'hash.

7.3.3 Script Spark-Cassandra

Per collegare Spark a Cassandra si utilizza il connettore fornito da DataStax. Come dataset di esempio viene considerato il file `u.user`, composto dai seguenti campi:

(`user_id`, `age`, `gender`, `occupation`, `zip`)

Un esempio di riga del dataset è il seguente:

(1, 24, M, writer, 85711)

¹Ricordati cosa è un hash nelle partizioni in SQL. Divide in parti uguali, ovvero distribuisce su più nodi in modo equo secondo una funzione matematica che prende un valore (in questo caso il `user_id`) e restituisce un numero "pseudo-casuale", ma sempre lo stesso per lo stesso input.

Osservazione: è fondamentale che i nomi dei campi letti dal file corrispondano esattamente ai nomi delle colonne presenti nella tabella di Cassandra, poiché Spark utilizza questa corrispondenza per il mapping dei dati.

La prima operazione consiste nel definire una funzione `parseInput`, che ha il compito di leggere una riga del file, suddividerla in campi e restituire un oggetto `Row` con i dati correttamente tipizzati. In Fig(7.3) è possibile osservare il pezzo di codice.

```
from pyspark.sql import SparkSession
from pyspark.sql import Row
from pyspark.sql import functions

def parseInput(line):
    fields = line.split('|')
    return Row(user_id = int(fields[0]), age = int(fields[1]), gender = fields[2], occupation = fields[3], zip = fields[4])
```

Figure 7.3: Lettura dati da u.user

Dopo aver definito la funzione di parsing, si procede con la creazione della sessione Spark. L'applicazione viene denominata `CassandraIntegration` e, tramite il metodo `config`, si specifica l'host a cui collegarsi per interagire con Cassandra (in questo caso `127.0.0.1`, cioè la macchina locale).

```
if __name__ == "__main__":
    # Create a SparkSession
    spark = SparkSession.builder.appName("CassandraIntegration").config("spark.cassandra.connection.host", "127.0.0.1").getOrCreate()

    # Get the raw data
    lines = spark.sparkContext.textFile("hdfs:///user/maria_dev/ml-100k/u.user")
    # Convert it to a RDD of Row objects with (userID, age, gender, occupation, zip)
    users = lines.map(parseInput)
    # Convert that to a DataFrame
    usersDataset = spark.createDataFrame(users)
```

Figure 7.4: Spark Session

In questa fase: `SparkSession.builder` avvia l'applicazione Spark e la configura per connettersi a Cassandra. `textFile()` legge il dataset come un insieme di stringhe, una per riga. `map(parseInput)` trasforma ogni riga in un oggetto `Row` con i campi tipizzati. Infine, con `createDataFrame`, si ottiene un `DataFrame` Spark utilizzabile per ulteriori trasformazioni o per scrivere i dati direttamente in Cassandra.

Una volta creato il `DataFrame`, è possibile salvarlo direttamente in Cassandra. L'operazione di scrittura si effettua specificando il formato di output, il modo di scrittura e i parametri relativi a keyspace e tabella di destinazione.

- `format("org.apache.spark.sql.cassandra")` indica a Spark di utilizzare il connettore Cassandra come destinazione dei dati.
- `mode('append')` definisce la modalità di scrittura: in questo caso i dati vengono aggiunti alla tabella esistente, senza sovrascriverla.
- `options(table="users", keyspace="movielens")` specifica i metadati fondamentali per Cassandra, ovvero la tabella `users` e il keyspace `movielens`.

```
# Write it into Cassandra
usersDataset.write\
    .format("org.apache.spark.sql.cassandra")\
    .mode('append')\
    .options(table="users", keyspace="movielens")\
    .save()
```

Figure 7.5: Scrivere in Cassandra

```
# Read it back from Cassandra into a new Dataframe
readUsers = spark.read\
    .format("org.apache.spark.sql.cassandra")\
    .options(table="users", keyspace="movielens")\
    .load()
```

Figure 7.6: Lettura da Cassandra a Spark

- `save()` avvia l'operazione effettiva di persistenza dei dati.

Una volta scritti i dati in Cassandra, è possibile leggerli nuovamente in Spark sotto forma di `DataFrame`. Il processo è simile a quello di scrittura, ma questa volta si utilizza l'API di lettura.

- `spark.read` avvia la lettura dei dati.
- `format("org.apache.spark.sql.cassandra")` dichiara che la sorgente è Cassandra.
- `options(...)` indica da quale tabella (`users`) e da quale keyspace (`movielens`) i dati devono essere prelevati.
- `load()` esegue effettivamente il caricamento e restituisce un nuovo `DataFrame`, qui memorizzato nella variabile `readUsers`.

7.4 MongoDB

MongoDB è un database non relazionale ampiamente utilizzato in ambito aziendale, in parte grazie al supporto fornito da una società dedicata. Oltre a gestire i dati, MongoDB offre una notevole flessibilità e presenta alcune caratteristiche distintive rispetto ad altri sistemi NoSQL. Come mostrato in Fig. 7.2, MongoDB si colloca in un punto intermedio tra consistenza e tolleranza alle partizioni, analogamente a HBase. Tuttavia, MongoDB utilizza un singolo nodo primario al quale devono essere indirizzate tutte le operazioni di scrittura per garantire la coerenza dei dati. Nel caso in cui il nodo primario non sia disponibile, il sistema attraversa un breve periodo di indisponibilità mentre viene eletto un nuovo nodo primario. Durante tale fase, è comunque possibile leggere i dati, ma le operazioni di scrittura vengono temporaneamente disabilitate.

Un altro punto di forza di MongoDB è la possibilità di memorizzare qualsiasi tipo di dato all'interno dei documenti. Un documento in MongoDB è un insieme strutturato di coppie chiave-valore, simile a un oggetto JSON, che può contenere dati di tipo scalare, array o documenti annidati. Questo modello consente una rappresentazione naturale e flessibile delle informazioni rispetto alle tradizionali tabelle dei database relazionali.

7.4.1 Documenti e Collezioni in MongoDB

In MongoDB, i dati non sono organizzati in tabelle come nei database relazionali, ma in *collezioni*, ciascuna delle quali contiene una serie di *documenti*. Un documento rappresenta un insieme strutturato di coppie chiave-valore, simile a un oggetto JSON, e può contenere dati scalari, array o addirittura altri documenti incorporati.

Ad esempio, un documento che rappresenta un post di un blog potrebbe avere la seguente struttura:

```
{
  "titolo": "Introduzione a MongoDB",
  "contenuto": "MongoDB è un database NoSQL...",
  "autore": "Mario Rossi",
  "commenti": [
    {
      "nome": "Luigi Bianchi",
      "email": "luigi@example.com",
      "testo": "Ottimo articolo!",
      "valutazione": 5
    },
    {
      "nome": "Anna Verdi",
      "email": "anna@example.com",
      "testo": "Molto utile, grazie!",
      "valutazione": 4
    }
  ]
}
```

In questo esempio, il campo `commenti` contiene un array di documenti incorporati, ciascuno dei quali rappresenta un commento associato al post.

MongoDB non richiede uno schema fisso: i documenti all'interno della stessa collezione possono avere campi diversi. È comunque possibile definire uno schema se si desidera applicare restrizioni sui dati, come nei database relazionali.

Ogni documento viene automaticamente dotato di un campo `_id` che funge da chiave primaria. È possibile creare indici su qualsiasi campo o combinazione di campi, rendendo le query più veloci. Per lo *sharding* orizzontale del database, è necessario indicizzare il campo su cui si desidera partizionare i dati.

7.4.2 MongoDB Architettura

La prima cosa da capire è il **Set di Repliche**. Come detto prima, MongoDB ha un singolo NameNode (master node) e se tale nodo cade, succede un casino. In MongoDB, la replica serve a garantire disponibilità e tolleranza per questo tipo di guasti. L'elemento fondamentale è il Replica Set, che è un gruppo di nodi (server) che contengono copie dello stesso database.

- All'interno del replica set c'è un unico nodo primario. Tutte le operazioni di scrittura (insert, update, delete) avvengono sul nodo primario. Questo nodo tiene traccia delle modifiche in un registro chiamato **oplog** (operation log).
- **Nodi secondari**: i nodi secondari ricevono automaticamente le operazioni registrate nell'oplog del primario. Essi mantengono copie aggiornate dei dati presenti nel nodo primario.
- **Failover automatico**: se il nodo primario fallisce, uno dei nodi secondari viene eletto automaticamente come nuovo primario.

Tuttavia non si sceglie un nodo secondario a caso, ma quel nodo dove il tempo di ping è più veloce e da quale è più reattivo.

Failover

Andiamo più in dettaglio su come funziona il failover. Innanzitutto controlla se l'utente ha richiesto una fonte di sincronizzazione specifica che puoi fare con qualcosa chiamato comando **replSetSyncFrom**. In tal caso, si sincronizza da quella fonte che hai specificato. Altrimenti itererà attraverso ogni nodo e controllerà alcune condizioni e infine selezionerà un nodo che ha il tempo di ping più basso e soddisfa tutte le condizioni.

Alcune di queste condizioni sono:

- **Maggioranza dei voti**: l'elezione del nodo primario si basa sul principio della maggioranza: più della metà dei nodi deve concordare su quale nodo diventerà primario. Questo significa che non puoi avere un numero pari di nodi "votanti", perché non si può mai ottenere una maggioranza chiara in caso di fallimento di un nodo.
- **Numero minimo di nodi**: il primo punto comporta che per avere replica e durabilità dei dati, almeno tre nodi sono necessari: due nodi secondari e uno primario. Questo è fonte di molta critica perché avere più nodi (più macchine) vuol dire spendere più soldi.
- **Nodo arbitro (Arbiter)**: per ridurre i costi senza sacrificare la capacità di elezione del primario, MongoDB permette di usare un nodo arbitro. L'arbitro non contiene dati; il suo unico compito è votare durante le elezioni del primario.

Un'altra problematica è che le applicazioni che usano Mongo devono sapere chi è il nodo primario e chi i secondari. E nel momento in cui i nodi cambiano (perché magari qualcuno cade) anche tutte le applicazioni devono sapere di questo cambiamento. Tuttavia, questo problema è stato affrontato in MongoDB 3.6 con una nuova funzionalità chiamata **DNS seedlist** che consente di concatenare i server a rotazione senza dover riconfigurare i client.

7.4.3 Secondary Delay

Il "secondo ritardato" è un nodo secondario all'interno del Replica Set che riceve gli aggiornamenti dal primario con un ritardo predefinito. Ad esempio, se imposti un ritardo di un'ora, i dati sul secondario saranno sempre "indietro" di un'ora rispetto al primario. Questo è davvero utile nel momento in cui si fanno errori stupidi e non vogliamo propagarli a tutti i nodi. Se, per esempio, cancelli accidentalmente un database sul primario, puoi recuperare i dati dal secondario ritardato prima che riceva quell'operazione distruttiva.

7.4.4 Sharding (partizionamento orizzontale)

Lo sharding è una tecnica di partizionamento orizzontale dei dati in un database distribuito. Così come abbiamo visto su HBase il partizionamento orizzontale tramite i server region, anche Mongo ha un suo tipo di partizionamento orizzontale.

Sharding significa suddividere un database molto grande in più "*pezzi*" chiamati **shard**, che vengono distribuiti su diversi server o nodi. Ogni shard contiene un sottoinsieme dei dati complessivi, ma tutti insieme formano il database completo.

MongoDB divide i dati usando una shard key: la shard key è un campo della collezione che decide come partizionare i documenti. MongoDB crea intervalli di valori della shard key chiamati **chunks**. Ogni chunk viene assegnato a uno shard. Ad esempio, se hai una collezione users con `user_id` come shard key:

- Chunk 1: `user_id` da 1 a 1000 lo metto nello shard A.
- Chunk 2: `user_id` da 1001 a 2001 lo metto nello shard B.
- E così via.

MongoDB monitora i chunks e sposta quelli più grandi o sbilanciati da uno shard all'altro. Questo meccanismo è chiamato balancer ed evita che un singolo shard si riempia troppo.

7.5 Come scegliere un DBMS

La scelta di un DBMS non è banale e dipende fortemente dal contesto applicativo. Un primo aspetto riguarda l'integrazione con i sistemi già in uso: tecnologie diverse offrono connettori e livelli di compatibilità differenti. Ad esempio, se l'analisi dei dati è affidata ad Apache Spark, sarà preferibile orientarsi verso database che garantiscano un'integrazione nativa o comunque semplice con Spark.

Un secondo criterio fondamentale è la capacità di scalare. Occorre valutare non solo la quantità di dati attuale, ma anche la crescita prevista nel tempo. Se il volume è potenzialmente illimitato, è opportuno considerare DBMS NoSQL come Cassandra, MongoDB o HBase, progettati per lo scaling orizzontale su cluster di nodi, in alternativa al tradizionale scaling verticale tipico dei database relazionali.

Un ulteriore criterio di scelta è legato al **teorema CAP**, secondo cui un sistema distribuito non può garantire contemporaneamente **Consistenza**, **Disponibilità** e **Tolleranza alle partizioni**. Se l'obiettivo principale è garantire la disponibilità del servizio, anche in caso di guasti temporanei, allora è necessario privilegiare la **Disponibilità** rispetto alla consistenza. In questi scenari

si accetta una *coerenza finale* (*eventual consistency*), dove una lettura immediata potrebbe restituire un valore obsoleto per pochi secondi. Questo approccio è tipico di molte applicazioni web ad alta scalabilità, dove la disponibilità è più critica della precisione istantanea dei dati.

Al contrario, in contesti come la gestione di transazioni finanziarie o dati sensibili, la priorità diventa la **Consistenza**, anche a costo di ridurre la disponibilità.

La regola generale è mantenere l'architettura il più semplice possibile: se i volumi di dati non richiedono una distribuzione su larga scala, è preferibile evitare la complessità di un database NoSQL distribuito.

7.5.1 Partizionamento verticale VS orizzontale

Il partizionamento di una tabella può essere effettuato secondo due approcci principali:

- **Partizionamento verticale:** la tabella viene divisa per colonne. Ogni partizione contiene tutte le righe ma solo un sottoinsieme degli attributi. È utile per separare dati frequentemente usati da quelli rari o pesanti (es. campi BLOB). Per ricostruire la tabella completa possono essere necessari *join*.
- **Partizionamento orizzontale:** la tabella viene divisa per righe. Ogni partizione contiene tutte le colonne ma solo un sottoinsieme di record (ad esempio in base a range di date o intervalli di valori). È molto usato per distribuire grandi quantità di dati su più nodi e supportare lo scaling orizzontale.

Esempio 1: Rubrica telefonica

Supponiamo di voler realizzare una semplice applicazione web per consultare numeri di telefono e indirizzi email dei colleghi. In questo scenario la **scala dei dati è limitata**: anche con migliaia di dipendenti è sufficiente un singolo disco.

Dal punto di vista della **consistenza**, non è critico se, a seguito di un aggiornamento, un utente visualizza ancora per pochi secondi un valore obsoleto. Inoltre, occorre privilegiare la **semplicità**: in molti casi il server web dispone già di MySQL installato, rendendo immediata l'integrazione. La scelta quindi è piuttosto ovvia: un database relazionale come **MySQL** soddisfa pienamente i requisiti, senza necessità di distribuire i dati o affrontare problematiche di tolleranza alle partizioni. L'unico requisito rilevante rimane garantire una buona disponibilità del servizio.

Esempio 2: Estrazione di registri web per analisi

Consideriamo il caso in cui si vogliano analizzare i log di accesso a un sito web per rispondere a domande come: «*qual è l'ora del giorno con il maggior numero di visite?*» oppure «*qual è la durata media delle sessioni?*».

Prima di introdurre un database non relazionale, è opportuno chiedersi se la scala del problema lo giustifica davvero. Se l'obiettivo è esclusivamente l'**analisi dei dati**, non serve un DBMS transazionale ad alte prestazioni: in questo caso entrano in gioco strumenti come **Hadoop** e **Spark**.

I dati di log possono essere importati in **HDFS** e analizzati in modalità **offline**. Qui non ci troviamo di fronte a scenari con altissimi tassi di transazioni o query ripetute in tempo reale, tipici invece delle applicazioni per cui sono stati progettati i database NoSQL.

Una volta che i dati risiedono nel cluster HDFS, è possibile sfruttare Spark per:

- strutturare e trasformare i log grezzi;
- eseguire analisi statistiche;
- applicare algoritmi di **machine learning** tramite la libreria **MLlib**.

In sintesi, introdurre un DBMS NoSQL in questo scenario sarebbe una complicazione inutile: la soluzione naturale è basarsi su strumenti di analisi distribuita come Hadoop e Spark.

Esempio 3: Sistema di raccomandazioni di film

Supponiamo di dover gestire un sistema che genera raccomandazioni personalizzate per gli utenti, ad esempio tramite un processo batch notturno basato su **Spark** o altri framework analitici.

Quando un utente accede al sito e richiede le proprie raccomandazioni, la risposta deve essere fornita in modo **rapido** e senza rallentare il rendering delle pagine web. In questo contesto le aziende attribuiscono grande importanza alla **disponibilità** del servizio e alle prestazioni in lettura, mentre possono tollerare un certo livello di **inconsistenza temporanea** (ad esempio leggere un set di raccomandazioni non ancora aggiornato).

Un database distribuito come **Apache Cassandra** è particolarmente adatto a questo scenario, poiché progettato per:

- garantire alta disponibilità e tolleranza ai guasti;
- scalare orizzontalmente su grandi volumi di utenti e dati;
- privilegiare le prestazioni in lettura/scrittura rispetto alla consistenza immediata.

In sintesi, quando la priorità è fornire risultati rapidi e sempre disponibili, accettando la *eventual consistency*, Cassandra rappresenta una scelta adeguata.

Esempio 4: Compravendita di azioni

Consideriamo un sistema di trading online, in cui possono essere presenti processi di analisi massiva in background su **Hadoop** o **Spark**, ma che necessita anche di un'interfaccia front-end per la compravendita in tempo reale.

In questo scenario la **tolleranza alle partizioni** è indispensabile, poiché il flusso di dati è enorme e continuo. Tuttavia, il requisito più critico è la **consistenza**: quando il valore di un'azione cambia, tutti gli utenti devono poterlo vedere praticamente in tempo reale. La **disponibilità** diventa quindi secondaria, accettando eventuali brevi interruzioni in caso di failover.

Sistemi come **HBase** o **MongoDB** sono adatti, poiché garantiscono coerenza tramite un modello a **master primario unico**. Questa architettura assicura che tutte le scritture passino attraverso un nodo centrale, mantenendo i dati consistenti, a scapito però della disponibilità: se il master si arresta, il cluster deve eleggere rapidamente un nuovo primario e aggiornare i client.

Tra le possibili scelte, un fattore discriminante è il livello di **supporto esterno** disponibile. MongoDB, ad esempio, è sostenuto da una forte comunità e da un'azienda che fornisce servizi di consulenza e sicurezza professionale, rendendolo spesso preferibile in contesti ad alta criticità come i sistemi finanziari.

Chapter 8

Controllo dei Cluster

8.1 YARN

YARN (Yet Another Resource Negotiator) è il sistema di gestione delle risorse di Hadoop. Il suo compito è coordinare e allocare in modo efficiente CPU e memoria sui nodi del cluster, permettendo a più applicazioni di convivere sullo stesso ambiente.

Dal punto di vista architetturale, YARN si colloca subito sopra **HDFS**. Mentre HDFS si occupa di distribuire lo **storage** dei dati, YARN distribuisce il **calcolo** necessario per elaborarli, cercando di mantenere la **località dei dati** (cioè far girare i processi sui nodi che già contengono i dati, minimizzando la comunicazione in rete).

8.1.1 Scheduler di YARN

Il cuore del funzionamento di YARN è il sistema di **pianificazione** dei job, che stabilisce come le risorse vengono assegnate. I principali scheduler sono:

- **FIFO (First In, First Out)**: esegue i job in ordine di arrivo, uno alla volta. È semplice e prevedibile, ma non sfrutta al meglio le risorse disponibili.
- **Capacity Scheduler**: consente l'esecuzione parallela di più job, assegnando le risorse inutilizzate ad altre applicazioni. È utile quando si vuole garantire un uso efficiente della capacità complessiva del cluster.
- **Fair Scheduler**: cerca di bilanciare le risorse tra job grandi e piccoli. Anche se un job pesante sta occupando l'intero cluster, un job leggero può comunque ottenere una quota di risorse per non restare bloccato.

Nella pratica, quando usiamo strumenti come **MapReduce**, **Spark**, **Hive** o **Pig**, stiamo già utilizzando YARN “sotto il cofano”. È YARN che decide dove e come far girare i processi, senza che l'utente debba gestire manualmente l'allocazione delle risorse.

Sebbene sia raro scrivere applicazioni direttamente su YARN, è possibile costruire livelli superiori che ne semplificano l'uso:

- **Apache Tez**: framework ottimizzato per eseguire grafi aciclici diretti (DAG), più efficiente di MapReduce.

8.2 Tez

Tez è un framework di esecuzione di grafi aciclici diretti (DAG) costruito sopra YARN. Fornisce un'alternativa più efficiente a MapReduce per elaborazioni batch o complesse, evitando di scrivere e leggere continuamente su HDFS tra le fasi intermedie. Invece di eseguire ogni fase come un job MapReduce separato, Tez costruisce un DAG delle operazioni da eseguire.

YARN gestisce le risorse e Tez coordina l'esecuzione dei task sul cluster seguendo il DAG.

8.2.1 Tez vs MapReducer

Partiamo dalla Fig(8.1). L'idea di base è che MapReduce accede al disco/HDFS più volte durante

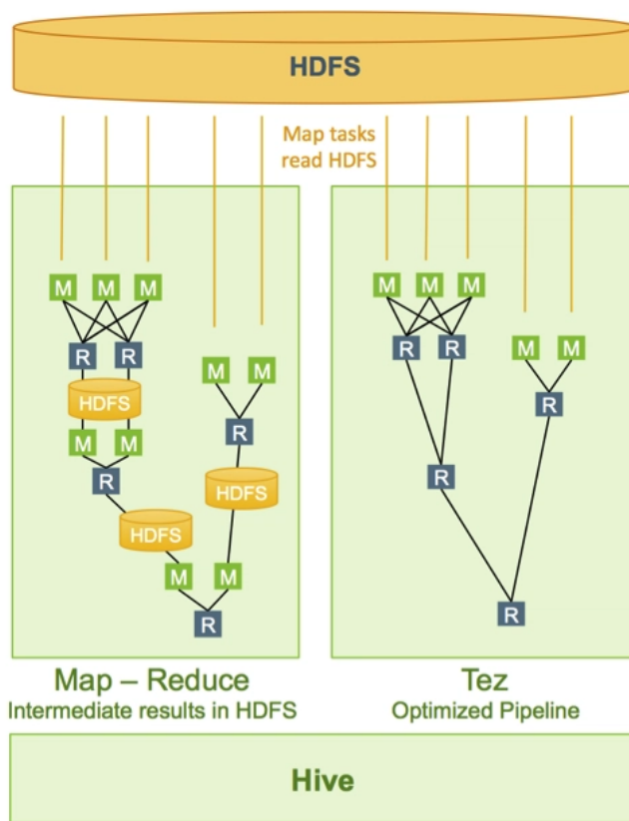


Figure 8.1: While MapReduce makes multiple reads/writes to HDFS, Tez avoids unneeded access to HDFS.

il flusso di dati, ovvero scrive e legge i dati (inizialmente puri da HDFS e poi modificati) durante ciascuna di queste fasi, con conseguenti 5-6 accessi al disco per un singolo processo MapReduce. D'altra parte, Tez recupera i dati dal disco, esegue fin da subito tutti i passaggi, memorizza i risultati intermedi in memoria, esegue la vettorializzazione (elabora batch di righe anziché una riga alla volta) e produce l'output.

In altre parole, mentre MapReduce esegue più letture/scritture su HDFS, Tez evita accessi non necessari a HDFS.

Ricordiamoci quindi che YARN è il negoziatore tra HDFS e le risorse come Sppark, MapReduce o Tez.

8.2.2 Hive con Tez

Hive può sfruttare Tez come motore di esecuzione al posto di MapReduce, migliorando notevolmente le prestazioni.

Per utilizzare Tez, è sufficiente modificare le impostazioni di Hive: cliccando sull'icona dell'ingranaggio accanto alla query, si aggiunge la proprietà

`hive.execution.engine` e si imposta il valore su `tez`.

A questo punto, l'esecuzione della query avviene tramite Tez: le operazioni vengono completate molto più rapidamente rispetto a MapReduce (ad esempio circa 20 secondi invece di oltre un minuto).

Inoltre, accedendo a **Tez View**, è possibile selezionare una delle operazioni eseguite e visualizzare, nella sezione *Graphical View*, il DAG (grafo aciclico diretto) costruito da Tez per orchestrare i task. Questo permette di comprendere come Tez suddivide e pianifica le operazioni sul cluster.

8.3 Mesos

Apache Mesos è un altro sistema di gestione delle risorse, simile a YARN, ma con un ambito più ampio. Mentre YARN è progettato specificamente per coordinare le risorse dei job Hadoop, Mesos gestisce le risorse di elaborazione a livello di **intero data center**, supportando diverse applicazioni e framework, anche al di fuori dell'ecosistema Hadoop.

Tecnicamente, Mesos non fa parte di Hadoop, ma è possibile integrare Hadoop YARN con Mesos tramite il progetto **Myriad**, che permette di eseguire job Hadoop su un cluster Mesos.

Mesos funziona come un **resource offer framework**: propone risorse disponibili ai framework registrati (ad esempio Spark o Storm), e spetta al framework decidere se accettare o rifiutare le risorse. Inoltre, l'algoritmo di scheduling viene definito dal framework stesso, offrendo grande flessibilità ma lasciando più responsabilità allo sviluppatore.

8.3.1 Mesos vs YARN

- **Ambito:** YARN è limitato ai job Hadoop, mentre Mesos gestisce risorse per qualsiasi applicazione distribuita nel data center.
- **Località dei dati:** YARN garantisce la **località dei dati** per i job MapReduce o Spark su HDFS, ottimizzando le prestazioni. Mesos, non essendo specifico per Hadoop, non gestisce automaticamente la località dei dati.
- **Flessibilità:** Mesos offre maggiore libertà nella gestione delle risorse e degli algoritmi di scheduling, adatto a Spark, Storm o altri framework non Hadoop.
- **Integrazione con Hadoop:** YARN è la scelta naturale se i dati risiedono su HDFS, perché coordina calcolo e storage sullo stesso cluster. Mesos può essere usato come alternativa, ma richiede configurazioni aggiuntive (ad esempio Myriad) per ottenere la stessa integrazione.

8.4 ZooKeeper

Apache ZooKeeper è un sistema centrale per il coordinamento e la gestione dello stato nei cluster distribuiti. La sua funzione principale è garantire che tutti i nodi del cluster abbiano

una **visione coerente** delle informazioni critiche, come il nodo master attivo o lo stato dei nodi secondari.

8.4.1 Ruolo nei sistemi distribuiti

Prendiamo come esempio **HBase**:

- Ogni client deve sapere quale nodo è il **master** attivo per inviare le scritture.
- ZooKeeper monitora lo stato di tutti i nodi e tiene traccia dei task assegnati ai lavoratori.
- In caso di fallimento di un nodo, ZooKeeper aiuta a decidere come redistribuire il lavoro e a quali nodi rivolgersi per continuare l'elaborazione senza perdita di coerenza.

Ad esempio, se il nodo master di HBase va giù, ZooKeeper:

- monitora i voti dei nodi per eleggere un nuovo master;
- controlla quali nodi secondari sono attivi o non disponibili;
- mantiene lo stato globale del cluster aggiornato per tutti i client.

8.4.2 Znodes

ZooKeeper organizza le informazioni in una struttura ad **albero**, simile a un filesystem, dove ogni elemento si chiama **znode**.

- Ogni znode può contenere dati e può avere figli, permettendo di rappresentare gerarchie di configurazioni o stati.
- Gli znodes sono utilizzati per memorizzare informazioni critiche come: il nodo master corrente, task assegnati o lo stato dei worker.
- Alcuni znodes possono essere *ephemeral*, cioè vengono rimossi automaticamente se il client che li ha creati si disconnette, utile per monitorare i nodi attivi.

8.4.3 Esempio pratico di ZooKeeper

Per comprendere meglio come funziona ZooKeeper, possiamo fare un piccolo esperimento pratico. Nella sandbox Hortonworks, ZooKeeper è già installato e configurato per partire automaticamente, quindi non è necessario avviare o installare nulla. Collegandoci al nodo con i permessi di root, possiamo navigare nella directory del client ZooKeeper e utilizzare il comando `zkCli.sh`, l'interfaccia a linea di comando, per connetterci al servizio.

Una volta connessi, possiamo esplorare la struttura ad albero di ZooKeeper, simile a un filesystem, con il comando `ls /`, che mostra i nodi principali già presenti per altri sistemi come HBase, Storm o Drill. Per creare un nostro nodo master, possiamo usare il comando `create -e /testmaster 127.0.0.1:2223`, dove l'opzione `-e` indica che il nodo è *ephemeral*: il nodo esiste solo finché la sessione che l'ha creato è attiva. Se la sessione termina, ZooKeeper elimina automaticamente il nodo, permettendo ad altri client di eleggere un nuovo master.

Possiamo verificare il contenuto di un nodo con `get /testmaster`, che restituisce ad esempio l'indirizzo del server master. Se il nodo master originale scompare, il nodo ephemeral viene rimosso e qualsiasi altro client può tentare di creare un nuovo nodo `/testmaster`. ZooKeeper garantisce che solo un client possa avere successo alla volta, assicurando così che esista sempre un solo master attivo.

In sintesi, tramite la creazione di znodes ephemeral e comandi come `create`, `get` e `ls`, ZooKeeper permette di mantenere la coerenza nel cluster, coordinare i master e monitorare lo stato dei nodi in modo semplice e affidabile, gestendo automaticamente i fallimenti senza intervento manuale.

8.5 Oozie

Apache Oozie è un sistema per l'esecuzione e la schedulazione di task Hadoop. Il concetto fondamentale in Oozie è il **workflow**, che permette di collegare diverse azioni all'interno di Hadoop in un flusso coerente.

Ogni workflow ha un **nodo iniziale** e un **nodo finale**, che indicano rispettivamente dove iniziare e dove terminare l'esecuzione. Tra questi, i **fork node** consentono di eseguire più azioni in parallelo. Ad esempio, un fork può avviare contemporaneamente un processo Pig per elaborare dati e un processo Sqoop per estrarre informazioni da un database esterno.

Una volta completate le azioni parallele, il workflow passa attraverso un **join node**, che attende che tutte le dipendenze siano terminate prima di procedere. Nell'esempio illustrato in Fig(8.2), sia Pig che Sqoop terminano il loro lavoro e inviano i risultati a un'azione Hive, che combina, filtra e organizza i dati in tabelle Hive finali.

Infine, una volta completata l'elaborazione di Hive, il workflow raggiunge il nodo di **end**, che indica la conclusione dell'intero processo. In questo modo, Oozie permette di orchestrare in modo efficiente flussi di lavoro complessi in Hadoop, gestendo sia le operazioni parallele che quelle sequenziali, garantendo che ogni step venga eseguito solo quando le dipendenze sono soddisfatte.

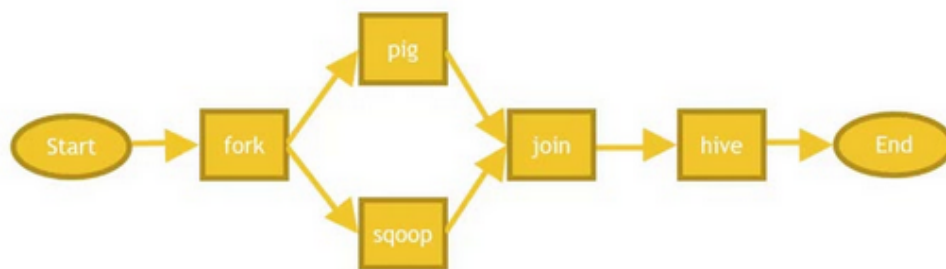


Figure 8.2: Esempio di un Workflow su Oozie.

Per far funzionare Oozie è importante sviluppare e testare tutto in anticipo, poiché il debug all'interno di Oozie non è semplice. Per configurare un flusso di lavoro, si crea una directory in HDFS contenente il file `workflow.xml`, che descrive tutte le specifiche del flusso. Sul disco locale della macchina da cui si avvia Oozie si prepara un file `job.properties`, dove si definiscono variabili a cui il flusso di lavoro può fare riferimento. In questo modo, lo stesso `workflow.xml` può essere utilizzato su server o directory diverse modificando solo le proprietà.

Il file delle proprietà tipicamente contiene informazioni come il nome host e la porta del NameNode HDFS, YARN, JobTracker, il nome della coda e il percorso di installazione di Oozie. All'interno del `workflow.xml` si possono usare variabili come `$nameNode` e `$jobTracker`, rendendo semplice aggiornare le informazioni del cluster senza modificare il flusso.

Per eseguire un lavoro, ci si collega al nodo master e si lancia il comando:

```
oozie job --oozie <URL_console_Oozie> -config <job.properties>
```

assicurandosi che la directory HDFS contenga il `workflow.xml`. La console web di Oozie permette di monitorare lo stato dei lavori in esecuzione, anche se non è possibile modificare nulla da lì.

Oozie offre inoltre i coordinatori, che consentono di pianificare l'esecuzione dei flussi su base periodica. Si può impostare un orario di inizio e una frequenza, ad esempio avviare un flusso alle 19:00 e ripeterlo ogni 24 ore.

Oozie 3.0 introduce il concetto di *bundle*, che è una raccolta di coordinatori gestibili insieme. Questa funzionalità è particolarmente utile nelle organizzazioni più grandi, dove diversi dipartimenti possono avere molti coordinatori che elaborano flussi di lavoro simili o dipendenti dagli stessi dati. Raggruppando i coordinatori in un bundle, è possibile sospendere o riattivare tutti i flussi contemporaneamente in caso di problemi operativi, come interruzioni nella raccolta dei dati, semplificando notevolmente la gestione.

Per fare un esempio pratico, si può impostare un flusso di lavoro Oozie in due fasi: prima utilizzare Sqoop per estrarre i dati dei film da un database MySQL e poi analizzarli con Hive. Questo semplice flusso mostra come collegare azioni Hadoop tramite Oozie e come i bundle possano aiutare a gestire più coordinatori in modo efficiente.

8.6 Zeppelin

Apache Zeppelin è uno strumento interattivo che permette di sperimentare rapidamente con script Apache Spark e visualizzare i dati in modi diversi. Oltre a Spark, Zeppelin supporta anche altri componenti dell'ecosistema Hadoop, come HBase, Cassandra, HDFS e Pig, tramite plugin dedicati. In pratica, si colloca come strato superiore nello stack Hadoop, offrendo un ambiente unificato per esplorare, analizzare e visualizzare i big data in modo interattivo.

Apache Zeppelin funziona in modo simile a JupyterLab: è un ambiente interattivo basato su notebook dove puoi scrivere codice, eseguirlo e vedere i risultati immediatamente. La differenza principale è che Zeppelin è progettato per l'ecosistema big data, quindi non serve solo per Python, ma supporta anche Spark (Scala, Python, SQL), Hive, Pig, HBase, Cassandra e altro. Puoi quindi fare analisi, trasformazioni e visualizzazioni su dati distribuiti direttamente nel cluster Hadoop.

8.7 Hue

Hue è una console di gestione dei cluster simile ad Ambari, ma è tipicamente associata a Cloudera, mentre Ambari è usato in Hortonworks. Così come Ambari, Hue fornisce un'interfaccia principale per amministrare e monitorare i servizi Hadoop.

La differenza principale tra Cloudera e Hortonworks è che Hortonworks è una soluzione completamente open source, mentre Cloudera integra alcune modifiche proprietarie nel sistema.

Una delle caratteristiche più utili di Hue è l'editor Oozie incorporato. Grazie a questo editor grafico, è molto semplice creare e gestire flussi di lavoro, coordinatori e bundle direttamente dal

browser, senza dover scrivere manualmente i file XML. In questo modo, impostare nuovi flussi di lavoro sul cluster e concatenare le varie attività diventa molto più rapido e intuitivo. Esiste anche una versione online di Hue che si può provare a questo indirizzo: <https://gethue.com>.

Chapter 9

Kafka & Flume

Lo **streaming di dati** è il processo di raccolta, elaborazione e analisi dei dati in tempo reale mentre vengono generati, invece di attendere che siano salvati in un database per elaborarli in seguito.

Immagina la situazione in cui un utente compie un'azione, per esempio un click su un prodotto su Amazon o un “like” su un social. Questa azione parte dal client, cioè dal browser o dall'app, e viene inviata al server. Di solito il passaggio avviene tramite una richiesta HTTP, oppure viene registrato come riga in un log, o ancora tramite i protocolli tipici dell'IoT se parliamo di sensori. A questo punto il server riceve l'informazione, ma non è detto che la scriva subito in un database tradizionale: i volumi sono troppo elevati e il rischio è di rallentare tutto il sistema. Entra allora in gioco un sistema di streaming, e tra i più usati c'è Apache Kafka. Il ruolo di Kafka è quello di fare da “cuscinetto” tra il server e i sistemi di archiviazione o analisi. Ogni evento prodotto dal client viene “pubblicato” su Kafka, che lo memorizza temporaneamente in un flusso ordinato, chiamato *topic*.

Un **topic** è come un grande raccoglitore dove vengono messi tutti i messaggi di uno stesso tipo. Per esempio, potresti avere un topic chiamato “*click_utenti*”, dove finiscono tutti i click fatti sul sito oppure un topic “*ordini*”, dove vengono inviati i dati sugli acquisti.

I topic sono ordinati e append-only: significa che i messaggi vengono sempre aggiunti in fondo, in ordine temporale, e mai cancellati subito. Ogni messaggio ha un suo numero progressivo (offset), che permette ai **consumer**¹ di leggerli in sequenza e ricordare fino a dove sono arrivati. La cosa potente è che più sistemi diversi possono leggere dallo stesso topic senza disturbarsi a vicenda: uno può analizzare i dati in tempo reale, un altro salvarli su HDFS, un altro ancora mandarli a un dashboard.

A differenza di un database, Kafka non ha lo scopo di conservare i dati per sempre, ma piuttosto di renderli disponibili in modo rapido e affidabile a chi li deve consumare. Da lì, infatti, i messaggi possono essere letti da diversi sistemi: qualcuno potrebbe analizzarli in tempo reale, ad esempio con Spark Streaming o Flink, per aggiornare subito le raccomandazioni di prodotti o alimentare una dashboard. Altri invece potrebbero salvarli in un database distribuito come HDFS, HBase o Cassandra, così da poterli conservare e analizzare in modalità batch. In questo modo il server non deve preoccuparsi di gestire direttamente scritture massive e simultanee,

¹Il consumer è semplicemente qualsiasi applicazione o servizio che ha bisogno di quei dati per fare qualcosa. Un consumer potrebbe essere il sistema che aggiorna in tempo reale i consigli personalizzati su Amazon: se hai cliccato un libro, ti propone altri libri simili.

perché delega tutto a Kafka, che diventa un punto centrale di raccolta e distribuzione dei dati. Questo modello permette di gestire enormi quantità di eventi, mantenendo il sistema scalabile e reattivo, e rendendo possibile sia l'analisi in tempo reale che l'archiviazione storica.

RIASSUNTONE: **HDP** è la **distribuzione** di Hortonworks che fornisce un ambiente per lavorare in Hadoop.

Hadoop è un **framework** open source, progettato per l'elaborazione e l'archiviazione distribuita di grandi quantità di dati. Cos'è un framework, è una raccolta di applicazioni riutilizzabili.

Hadoop è dotato di un filesystem, un resource manager e di un motore per l'elaborazione dei dati.

Il **filesystem** si chiama **HDFS** (Hadoop Distributed File System), come dice il nome è un file system distribuito cioè suddivide un file in blocchi e questi blocchi vengono salvati in un cluster formato da 1 o più nodi (macchine). I blocchi vengono replicati su più nodi in modo da garantire un backup in caso di crash di un nodo. Inoltre la distribuzione dei blocchi in più nodi permette l'esecuzione dei calcoli in parallelo.

Il **resource manager** si chiama **YARN** (Yet Another Resource Negotiator). Quello che fa è occuparsi di gestire le risorse del cluster (CPU, RAM ecc.) e di schedulare i job inviati dagli utenti, come se fosse un sistema operativo.

Il **motore di elaborazione** dei dati invece è il **MapReducer**. Non è altro che un modello di programmazione che si basa su due concetti:

Def motore di elaborazione: componente software che si occupa di prendere un lavoro, scomporlo in task distribuiti, inviare i task ai nodi del cluster, coordinare l'esecuzione e alla fine raccogliere i risultati.

mapping: trasforma/elabora i dati in parallelo

reducing: aggrega i risultati del mapping in un unico risultato.

Tra mapping e reducing Hadoop ordina i dati e raggruppa i valori in base alla chiave. Le funzioni di mapping e reducing vengono definite dall'utente.

MapReduce è implementato principalmente in Java e per questo motivo le funzioni sono più veloci se scritte in Java. è però possibile scrivere le funzioni anche in Python.

Un'altra alternativa a Java è il linguaggio Pig Latin, introdotto da Pig. Quello che fa è semplificare la scrittura di MapReduce con un linguaggio simile a SQL.

Invece un'alternativa più veloce e semplice di MapReduce è **SPARK**. Anche Spark è un **motore di elaborazione** dei dati distribuito. è scalabile, nel senso che sfrutta un cluster di più macchine per eseguire operazioni in parallelo. è il software responsabile dell'esecuzione efficiente e parallela di programmi che elaborano dati su cluster.

Si basa su un concetto: **RDD (Resilient Distributed Dataset)**. Questo significa che: i dati vengono divisi in blocchi e distribuiti su diversi nodi e ogni blocco ha una copia su ogni nodo, in questo modo è garantita la resilienza dei dati, cioè in caso di crash di un nodo ho sempre una copia da cui prendere quella porzione di dati.

Utilizza trasformazioni e azioni, le prime modificano l'RDD creandone delle sottoversioni ad ogni modifica. Le seconde attivano l'esecuzione di tutte le trasformazioni fatte fino a quel momento. Al verificarsi di un'azione Spark costruisce un grafo diretto aciclico (DAG) che tiene traccia di tutte le trasformazioni fatte per trasformare i dati. Poi trova il path migliore per arrivare a quel risultato e distribuisce i task agli executor.

Hive è il sistema di **datawarehousing** di Hadoop. Cioè è il sistema che permette di organizzare, interrogare e analizzare grandi quantità di dati.

Un sistema come Hive permette di avere una serie di tabelle, partizioni e schemi sui dati su cui è possibile eseguire query SQL-like.

Offre quindi il vantaggio di non dover scrivere le funzioni di MapReduce in Java.

Hive può usare Spark come motore di elaborazione dei dati, per eseguire query in parallelo.